

HEC MONTRÉAL

**AI-Powered Tourism Intelligence Pipeline for Destination Québec
cité:**

Automated Media Monitoring, NLP Classification, and Sentiment Analysis

par

[Student Name]

[Supervisor Name]

HEC Montréal

Directeur/Directrice de recherche

Sciences de la gestion

(Spécialisation Business Analytics / Intelligence d'affaires)

Projet supervisé présenté en vue de l'obtention

Abstract

This project describes the development and deployment of an automated media monitoring intelligence pipeline, built for Destination Québec cité (DQc), the organization responsible for promoting tourism across the Québec Capitale-Nationale region. The goal is straightforward: turn the daily flood of news coverage about Quebec City into structured, usable data that decision-makers can actually act on.

Each day, the pipeline pulls articles from over thirty francophone and anglophone media sitemaps, along with a supplementary news search interface. Every article then moves through a nine-stage workflow. The system starts by resolving URLs and scraping full article text using a headless browser set up to handle dynamic pages. From there, a filtering stage checks whether each article is genuinely relevant to the region, using a rubric built around the specific municipalities, landmarks, and tourism assets that matter to DQc. Anything off-topic gets dropped early.

The articles that make it through are then enriched in several ways. The pipeline scores sentiment, assigns standardized news categories, and tags each piece with domain-specific labels covering attractions, hotels, and restaurants. It also groups related articles into clusters based on how similar they are in meaning, which makes it possible to follow a single story as it spreads across different outlets. At the end of the chain, each article gets a short summary along with geographic tags and an assessment of how the coverage might shape visitor perception. Everything the pipeline produces is stored in a cloud data warehouse and feeds into a monitoring Looker dashboard. The whole system runs daily in a containerized cloud environment with no manual intervention needed. In practice, it gives DQc a continuously updated picture of how Quebec City's tourism sector is being covered in the press, helping the team spot emerging stories, track public sentiment, and get ahead of both opportunities and risks.

Keywords: Natural Language Processing, Tourism Intelligence, News Monitoring, Data Pipeline, Cloud Computing, Sentiment Analysis, Text Classification, Web Scraping

Table of Contents

Abstract

Introduction

- The Organization: Destination Québec cité
- Project Mandate
- Report Structure

Literature Review

- Web Scraping and Automated Data Collection
- Transformer-Based NLP Models
- Text Classification Taxonomies and Zero-Shot Learning
- Large Language Models for Information Extraction
- Sentence Embeddings and Document Clustering

Chapter 1: Methodology and Technical Implementation

- Overall Architecture: The Nine-Stage Pipeline
- Stage 1: Sitemap Scraping and Article Discovery
- Stage 2: Gemini Relevance Analysis
- Stage 3: Playwright Web Scraping
- Stage 4: Sentiment Analysis
- Stage 5: Semantic Article Clustering
- Stage 6: IPTC Topic Classification
- Stage 7: GLiClass Multi-Label Classification
- Stage 8: Gemini Summarization and Perception Analysis
- Stage 9: BigQuery Data Persistence

Chapter 2: Results and Analysis

- Pipeline Performance Overview
- Stakeholder Feedback and Organizational Impact

Chapter 3: Knowledge Transfer

- Technical Documentation
- Prompt Documentation and Maintenance Guide
- Dashboard and End-User Training
- Operational Handover

Conclusion

- Limitations
- Future Directions
- Personal Reflection

Bibliography

Annexes

- Annex A: Pipeline Configuration Parameters

Annex B: IPTC French Label Mapping

Annex C: Gemini Relevance Scoring Examples

Introduction

The Organization: Destination Québec cité

This supervised project was carried out within Destination Québec cité (DQc), the official organization responsible for promoting tourism in the Capitale-Nationale region of Quebec. DQc acts as the bridge between public authorities, hotels, restaurants, attractions, event organizers, and other hospitality businesses that together make Quebec City a major travel destination. Its work covers four main areas: communications and branding, event promotion, relationships with member businesses, and data-driven support for tourism planning and investment.

DQc's territory goes well beyond Quebec City itself. It covers the full city agglomeration—including boroughs like La Cité-Limoilou, Sainte-Foy–Sillery–Cap-Rouge, Beauport, and Charlesbourg, along with nearby municipalities such as L'Ancienne-Lorette, Saint-Augustin-de-Desmaures, and the Indigenous community of Wendake. It also takes in four surrounding regional county municipalities (MRCs): La Jacques-Cartier (ski resorts, Village Vacances Valcartier), La Côte-de-Beaupré (Mont-Sainte-Anne, Montmorency Falls, Canyon Sainte-Anne), L'Île-d'Orléans (vineyards, cideries, heritage farms), and Portneuf (Vallée Bras-du-Nord, the Chemin du Roy). The region pulls in billions of dollars in tourism revenue and welcomes millions of visitors each year, which makes keeping a close eye on media coverage and public perception a real priority for the organization.

When this project started, DQc was still handling media monitoring mostly by hand. A small team of analysts would scan news sites, Google Alerts, and social media each morning looking for articles that touched on the region's tourism interests. DQc also had partnerships with external news gathering services such as Référence Média and others, which provided curated media clippings for a fee. However, even with these paid services, the actual work still relied heavily on manual searches by employees, and the scope of what was being captured remained too narrow. The regional media landscape had grown to include dozens of digital-only publications, multilingual national outlets, and international travel media, all publishing around the clock. With no centralized system to collect, organize, and analyze all

that content, important articles were regularly being missed, emerging trends went unnoticed until they turned into problems, and DQc's leadership had no reliable, data-backed full picture of how Quebec City was actually being portrayed in the press.

Project Mandate

The mandate for this supervised project was defined in collaboration with DQc's management team and formalized at the outset of the engagement. The core objective was to design, build, and deploy an artificial intelligence system capable of automating DQc's media intelligence workflow, replacing the manual, ad hoc process with a structured, repeatable, and scalable pipeline that would operate continuously with minimal human intervention. Specifically, the mandate comprised the following deliverables:

1. **Automated Data Collection:** Build a system to automatically discover and collect news articles relevant to Quebec City's tourism sector from multiple francophone and anglophone media sources on a daily basis, covering local, regional, provincial, national, and international outlets.
2. **Relevance Filtering:** Develop an intelligent filtering mechanism that can distinguish articles with genuine impact on Quebec City's tourism economy from the vast majority of general news that merely happens to mention the word "Québec," accounting for the critical ambiguity between Quebec-the-city and Quebec-the-province.
3. **NLP Analysis:** Apply natural language processing techniques to classify articles by topic, assess their sentiment, identify semantic clusters of related coverage, and assign domain-specific tourism labels (attractions, lodging, restaurants).
4. **Summarization and Perception Assessment:** Generate concise, human-readable summaries of each article along with an assessment of its impact on visitor perception of Quebec City as a destination.
5. **Data Infrastructure:** Persist all analytical outputs in a cloud data warehouse (Google BigQuery) to serve as the foundation for a real-time monitoring dashboard accessible to DQc's management team and analysts via Looker.

6. **Deployment:** Containerize and deploy the pipeline on Google Cloud Platform for automated, scheduled execution without ongoing technical intervention.

The strategic objective behind this mandate was to reinforce DQc's analytical and anticipatory capabilities by automating its sectoral media monitoring, thereby enabling faster, better-informed decisions in matters of communication strategy, event planning, member engagement, and crisis response. The project was explicitly positioned not as a research prototype but as a production system intended for daily operational use by non-technical staff.

Literature Review

This section surveys the key research, tools, and models that informed the design of the pipeline. Each subsection briefly introduces the relevant literature and explains why a particular approach or model was adopted for this project.

[1] Barbaresi, A. (2021). Trafilatura: A web scraping library and command-line tool for text discovery and extraction. *Proceedings of ACL 2021 System Demonstrations*.

<https://aclanthology.org/2021.acl-demo.15.pdf>

Relevance: Selected as the pipeline’s HTML content extraction engine for Stage 3 (web scraping). Trafilatura outperformed alternatives such as newspaper3k and readability-lxml in extraction accuracy and offered strong multilingual support essential for processing both francophone and anglophone news sources.

[2] Barbieri, F., Camacho-Collados, J., Espinosa Anke, L., & Neves, L. (2022). TweetEval: Unified benchmark and comparative evaluation for tweet classification. *Findings of EMNLP 2020*. <https://aclanthology.org/2020.findings-emnlp.148/>

Relevance: Underpins the CardiffNLP twitter-xlm-roberta-base-sentiment model used in Stage 4 (sentiment analysis). This model was chosen over lexicon-based and traditional ML approaches because it provides robust bilingual (French/English) sentiment classification out of the box, with no domain-specific fine-tuning required—critical in a data-scarce context where no labeled Quebec tourism media dataset exists.

[3] Kuzman, T., & Ljubešić, N. (2025). Multilingual news topic classification with the IPTC standard. *IEEE Access*. <https://ieeexplore.ieee.org/document/10900365>

Relevance: Validates the classla/multilingual-IPTC-news-topic-classifier used in Stage 6. This study demonstrates that the model achieves macro-F1 of 0.746 and micro-F1 of 0.734 on a multilingual test set, outperforming GPT-4o in a zero-shot setting—confirming it as a strong, cost-effective choice for production-level news classification.

[4] Brown, M. A., Gruen, A., Maldoff, G., Messing, S., Sanderson, Z., & Zimmer, M. (2025). Web scraping for research: Legal, ethical, institutional, and scientific considerations. *Big Data & Society*. <https://arxiv.org/pdf/2410.23432>

Relevance: Provided the most comprehensive and up-to-date legal analysis for web scraping in a research context, covering CFAA case law (hiQ Labs v. LinkedIn, Meta v. Bright Data), GDPR obligations, copyright fair use doctrine, and institutional review board considerations. This paper complemented the earlier Krotov and Silva (2018) framework by addressing the post-2020 legal landscape—including the impact of AI-driven scraping on copyright norms—and directly informed the pipeline’s data collection policies for Stage 1.

[5] Díaz-Pacheco, Á., Guerrero-Rodríguez, R., Álvarez-Carmona, M.Á., Rodríguez-González, A.Y., & Aranda, R. (2023). A comprehensive deep learning approach for topic discovering and sentiment analysis of textual information in tourism. *Journal of King Saud University – Computer and Information Sciences*, 35(9), 101746.

<https://www.sciencedirect.com/science/article/pii/S1319157823003002>

Relevance: Inspired the pipeline’s approach to semantic article clustering (Stage 5). Díaz-Pacheco et al. demonstrated that combining transformer-based document embeddings with clustering algorithms effectively discovers latent thematic structures in tourism-related text, significantly outperforming traditional bag-of-words methods. This validated the pipeline’s decision to use dense multilingual embeddings (BAAI/bge-m3) with cosine similarity and connected-component clustering rather than keyword-based approaches like TF-IDF.

[6] Zhao, T., Meng, L., Song, D., et al. (2022). Aspect-based sentiment analysis using local context focus mechanism with DeBERTa. *arXiv:2207.02424*.

<https://arxiv.org/abs/2207.02424>

Relevance: Provides the architectural foundation for the DeBERTa-based ABSA model used in the aspect-level sentiment analysis stage. The paper details how DeBERTa’s disentangled attention combined with a Local Context Focus mechanism enables aspect-conditional sentiment prediction, where the same input text yields different polarities depending on the aspect term paired with it. Benchmarks across SemEval domains validate the model’s ability to capture fine-grained, topic-dependent sentiment.

[7] International Press Telecommunications Council. (2024). IPTC NewsCodes.

<https://iptc.org/standards/newscodes/>

Relevance: Defines the standardized seventeen-category news topic taxonomy adopted in Stage 6 (IPTC topic classification). Using an industry-standard taxonomy ensures that the pipeline's topic labels are interpretable by media professionals and compatible with external news intelligence tools.

[8] Moń, M., & Pańczyk, B. (2025). A comparative analysis of web application test automation tools. *Journal of Computer Sciences Institute*, 35, 159–165.

<https://ph.pollub.pl/index.php/jcsi/article/download/7119/5017>

Relevance: Provides peer-reviewed empirical evidence supporting the selection of Playwright as the browser automation framework for Stage 1 (sitemap parsing) and Stage 3 (dynamic content extraction). The study compared Playwright, Cypress, and Selenium across average test execution time, CPU usage, and RAM consumption, concluding that Playwright demonstrated higher effectiveness and flexibility in most test cases due to its modern architecture and features such as browser context isolation, network interception, and multi-language support — all directly applicable to scraping JavaScript-heavy news sites.

[9] Günther, M., Mohr, I., Wang, B., & Xiao, H. (2024). Late chunking: Contextual chunk embeddings using long-context embedding models. *arXiv:2409.04701*.

<https://arxiv.org/abs/2409.04701>

Relevance: Provides the methodological foundation for the chunking and embedding strategy used in Stage 5 (semantic article clustering) and any RAG-style retrieval components in the pipeline. The paper demonstrates that traditional recursive text splitting and semantic chunking lose cross-chunk context because each chunk is embedded in isolation, leading to degraded retrieval performance, particularly for documents containing pronouns, definite references, or entity mentions that depend on surrounding text. By contrast, late chunking embeds the entire long-form document first using a long-context embedding model (e.g., jina-embeddings-v2 with 8,192-token capacity) and apply chunk boundaries only after token-level embeddings are produced. This preserves the full document context within each chunk's

embedding and consistently outperforms naive chunking across BEIR retrieval benchmarks, with larger gains observed on longer documents, a property directly relevant to news articles in the pipeline, which often span 500–2,000 words.

[10] Zaratiana, U., Tomeh, N., Holat, P., & Charnois, T. (2023). GLiNER: Generalist model for named entity recognition using bidirectional transformer. arXiv:2311.08526.
<https://arxiv.org/abs/2311.08526>

Relevance: Provides the architectural and empirical foundation for the GLiNER-x-large model used in the named entity recognition stage of the pipeline. Unlike traditional NER models such as Facebook's BART-MNLI or fixed-class BERT-based taggers, GLiNER reframes NER as a span-matching task between text tokens and natural-language entity-type prompts, processed through a single bidirectional encoder. This enables true zero-shot recognition of arbitrary entity types specified at inference time without retraining. The paper demonstrates that GLiNER outperforms ChatGPT and fine-tuned LLMs in zero-shot evaluations across 20+ public NER benchmarks while remaining lightweight (50M–300M parameters) and CPU-deployable. The bidirectional encoder also enables parallel entity extraction, avoiding the slow sequential token generation of autoregressive LLMs — a property critical for high-volume news article processing.

[11] Stepanov, I., Shtopko, M., Vodianytskyi, D., Lukashov, O., Yavorskyi, A., & Yaroshenko, M. (2025). GLiClass: Generalist lightweight model for sequence classification tasks. arXiv:2508.07662. <https://arxiv.org/abs/2508.07662>

Relevance: Provides the architectural foundation for the GLiClass-x-base model used for multi-label tourism classification (attraction, lodging, restaurant). GLiClass adapts the GLiNER paradigm to sequence-level classification, treating labels as natural-language prompts encoded jointly with input text in a single forward pass. This makes it functionally equivalent to a cross-encoder while remaining up to 50× faster, since classification scores for all candidate labels are produced simultaneously rather than through repeated text-label pair evaluations. Compared to standard zero-shot baselines such as Facebook BART-MNLI (which requires one forward pass per label) or fixed-class BERT classifiers (which require full retraining for new label sets), GLiClass offers superior throughput and zero-shot

flexibility, making it well-suited for evolving label taxonomies in production NLP pipelines. The paper also demonstrates strong few-shot performance, justifying the pipeline's decision to fine-tune GLiClass-x-base with QLoRA on a small hand-crafted bilingual dataset rather than train a classifier from scratch.

Chapter 1: Methodology and Technical Implementation

This chapter provides a detailed description of the methodology adopted for the pipeline, organized around its nine sequential stages. Where challenges were encountered that required changes in approach, these are documented explicitly, along with the reasoning behind the pivots.

Overall Architecture: The Nine-Stage Pipeline

The pipeline was designed as a sequential, nine-stage data processing workflow where each stage transforms or enriches a shared data structure (a pandas DataFrame) before passing it to the next stage. The entire pipeline runs as a single containerized job on a daily schedule, the data volumes are moderate (hundreds of articles per day, not millions), and the linear design is dramatically simpler to debug, monitor, and maintain for a small team. The nine stages are:

Stage	Name	Technology	Purpose
1	Sitemap Scraping	requests, xml.etree, SerpAPI	Discover articles from past 24 hours
2	Filtering/Relevance Analysis	Gemini LLM	Score relevance to QC tourism (0–100)
3	Web Scraping/Browser automation	Playwright, Trafilatura	Extract full article text
4	Sentiment Analysis	twitter-xlm-roberta	Compute sentiment score (–1 to +1)
5	Clustering	BAAI/bge-m3 + cosine sim	Group duplicate/related articles
6	IPTC Classification	multilingual-IPTC classifier	Assign standardized news topic
7	Domain Classification	GLiClass fine tuned	Score Attrait/Hebergement/Restaurant

8	Summarization	Gemini LLM + Pydantic	Summary, perception, geographic tag
9	Data Storage + vizualization	Google BigQuery	Persist to cloud data warehouse

Stage 1: Sitemap Scraping and Article Discovery

The pipeline's first stage discovers all news articles published within the preceding 24 hour window. It parses XML sitemaps and RSS feeds from a curated list of over twenty Canadian media sources, supplemented by programmatic Google News searches via SerpAPI.

The Need to Build a Collection System from Scratch

One of the biggest challenges I faced upfront and early on was that DQC had no existing dataset to work with, no internal archive, no structured media database, nothing. The original vision was to leverage existing data feeds, APIs, or aggregated datasets to populate the intelligence pipeline, but in practice, no such resources existed. Quebec's regional media ecosystem does not offer centralized API access, and DQC had no pre-existing data collection infrastructure or historical article databases. This meant that a significant portion of the project's development time, originally allocated for NLP model fine-tuning and dashboard refinement, had to be redirected toward building the entire data collection layer (Stages 1 to 3) from the ground up. The sitemap parser, the SerpAPI integration, the Google News URL resolver, and the Playwright scraper with browser automation collectively represent a substantial engineering effort that was not originally anticipated in the project scope but proved essential for the system's viability.

Legal and Ethical Framework

I started by researching the legal side of web scraping. Krotov and Silva (2018) provide a useful framework for thinking about the ethical dimensions, and Mitchell (2018) covers the practical techniques in Python. After presenting my findings to the team, we agreed on a clear boundary: publicly available news content containing no personal information could be collected for internal analytical use, as long as we weren't redistributing it commercially or profiting from the content itself, as it can violate terms and conditions of some websites.

Choosing an Approach

The next question was where to start. I couldn't just download the entire web and analyze everything, that's not feasible in terms of time, cost, or computing resources. I needed a focused, efficient approach. Since I was new to scraping at this scale, I spent a lot of time exploring different methods. I watched tutorials, read documentation, and experimented. My first instinct was to build a traditional web crawler, but I quickly realized after testing the code how slow and unreliable that approach was in practice, especially for a production system that needed to run daily. Also, due to the fact that the website's servers would be bombarded, we might get rate limited pretty easily.

I also looked into commercial news aggregator APIs like NewsAPI and NewsCatcher, and I proposed a few of these to the team as an initial data access point to build the NLP pipeline upon it. But due to the monthly enterprise package cost being somewhat out of budget for the company, and the fact that I didn't want the company to commit to paid subscriptions before the NLP pipeline was even built and proven, it felt like putting the cart before the horse.

So I kept looking for alternatives, and that's when I discovered something that changed the direction of the project: nearly every news website publishes XML sitemaps or RSS feeds. These are machine-readable files that websites create specifically so that search engines (like Google for example) can find and index their content. For the Quebec media outlets I was targeting, most of which don't offer any kind of API, these sitemaps turned out to be a goldmine. Parsing them gave me a respectful, lightweight way to discover new articles every day, since these feeds are explicitly intended for machine consumption.

One downside I discovered is that the sitemaps give only the URLs containing the title of the news articles and we cannot access the content. Also some sites don't have readable sitemaps, either poorly formatted, outdated, or not readable by Python libraries.

Sitemap Source Configuration

For the XML part, I built a curated list of over twenty Canadian media sources spanning both languages. I had to research the major relevant news media myself, as the company did not

have an internal list (only limited to a few websites). The monitored sitemaps encompass major francophone and anglophone outlets:

Source Domain	Format	Language
La Presse (lapresse.ca)	XML Sitemap	French
Radio-Canada (ici.radio-canada.ca)	XML Sitemap	French
TVA Nouvelles (tvanouvelles.ca)	XML Sitemap	French
Journal de Québec (journaldequebec.com)	XML Sitemap	French
Le Soleil (lesoleil.com)	XML Sitemap	French
Le Devoir (ledevoir.com) — 5 sections	RSS Feed	French
La Tribune (latribune.ca)	XML Sitemap	French
24 Heures (24heures.ca)	XML Sitemap	French
Carrefour de Québec	RSS Feed	French
Journal de Montréal	XML Sitemap	French
Global News (globalnews.ca)	XML Sitemap	English
Montreal Gazette (montrealgazette.com)	XML Sitemap	English
National Post (nationalpost.com)	XML Sitemap	English
Globe and Mail (theglobeandmail.com)	RSS Feed	English
Narcity (narcity.com)	XML Sitemap	English
MTL Blog (mtlblog.com)	XML Sitemap	English

Filling the Gaps with Google News and SerpAPI

I discovered that some sites don't have comprehensive readable sitemaps, either poorly formatted, outdated, or not readable by Python libraries, and some with no sitemaps at all, which would require other web crawling techniques that would be too hard to implement. But I wanted the media review to be broad and have a high coverage for the company, so I researched other ways I could add on to the XMLs of the 20 to 25 major websites I already had.

To fill in the gaps for outlets that didn't have well-structured sitemaps, I first tried to capture the RSS feed of a news aggregator and I was able to discover the Google News RSS available for personal use under their terms. I built my original pipeline by manually parsing the RSS like I did for other websites but had to use the `gnewsdecoder` Python library to decode the website URLs that Google encrypts in their RSS feed.

Later on, I used SerpAPI-mediated Google News searches as a secondary discovery method because they did what I do manually (parsing Google News RSS among others like Bing etc.), because I realized it was much faster as their servers were much quicker and more efficient than my code. I hit rate limit 503 errors because decoding encrypted URLs counted as visiting the page. I tested putting delays but due to the hundreds of URLs, it would slow down the pipeline significantly, which I didn't want (because Google Cloud charges on the time the pipeline runs).

SerpAPI offered 250 free searches per month, so I implemented six search configurations covering both languages, Canadian and international perspectives, and domain-restricted searches for niche outlets like `tourismexpress.com`, `monvieuxquebec.com`, and `urbania.ca`, since it was not a good idea to parse the whole RSS for those because the percentage of Quebec tourism news was less than 5% for those sites. Google RSS has built in filtering functions (which I discovered how to manage while manually parsing Google RSS), to narrow the search and retrieve more niche URLs closely related to Quebec tourism.

Parsing Google RSS manually gave me a good intuition on how to use their filters to access the most news possible, among which using the least possible vocabulary for a query produces the most results because it does not restrict the search. I figured out though that SerpAPI does not filter Google RSS well, so I had to implement code to keep only the articles from the past 24 hours so as not to analyze the same articles each day. I did so for all other websites too, as RSS feeds cover more than 24 hours of news.

A global budget of eight API calls per run keeps costs under control at \$0. This hybrid approach gave me broad coverage without depending on any single source or paid service.

Parsing and Deduplication

Each sitemap URL is fetched concurrently using a `ThreadPoolExecutor` with five worker threads. The parser handles two XML formats: RSS feeds, identified by the `<rss>` root element, extracting `<link>`, `<pubDate>`, and `<title>` from each `<item>`; and standard XML sitemaps, iterating over `<url>` elements and extracting `<loc>`, `<lastmod>` or `<news:publication_date>`, and `<news:title>`.

Articles discovered from multiple sources are deduplicated by normalizing URLs, stripping tracking parameters like `utm_source`, `utm_medium`, `utm_campaign`, `fbclid`, and `gclid`, and applying exact title matching as a fallback. This dual approach ensures only unique articles proceed to the next stage. This approach helped not to waste resources by analyzing the same articles each day, by filtering from the last 24 hours among the RSS feeds' URLs and making sure we deduplicate same news either by URL or if it has exact same title wording, to avoid analyzing identical articles.

Figure 5: HTTP 403 Forbidden error from news sites

HTTP 403 Forbidden Errors and User-Agent Engineering

The earliest scraping attempts used Python's `requests` library with default headers, no User-Agent string, no `Accept` headers, no language preferences. Multiple major outlets, including TVA Nouvelles (tvanouvelles.ca), immediately returned HTTP 403 Forbidden errors, as their servers were configured to reject requests that did not present browser-like headers. The fix was to add a realistic User-Agent header mimicking a common browser (Chrome on Windows), along with `Accept`, `Accept-Language`, and `DNT` headers. This single change resolved 403 errors on most sitemaps, but it revealed a deeper issue: each subsequent problem with a different outlet required additional header refinements, eventually leading to the pool of seven randomized User-Agent strings and the matching `sec-ch-ua` header generation system present in the final pipeline.

The Timezone Parsing Nightmare

Figure 11: Timezone parsing challenges across sources

Date parsing proved far more challenging than initially anticipated. Each media outlet formats timestamps differently: some use ISO 8601 (2025-11-25T14:28:00+00:00), some use RFC 2822 (Sat, 29 Nov 2025 14:28:00 GMT), some use informal formats (Nov 17, 2025, 12:52 PM), and SerpAPI returns relative dates ("2 hours ago"). Timezone handling introduced additional complexity. A Bing News item was observed with a "GMT" timezone suffix that Python's standard datetime parser could not handle, requiring a dedicated parsing pathway through `email.utils.parsedate_to_datetime()`. The final date parser supports over twelve distinct format patterns and three parsing strategies (email-style, ISO, and `strptime`), all normalizing to UTC before comparison. This parser was developed incrementally over several weeks, with each new source revealing a previously unseen date format that caused articles to be incorrectly excluded from the 24-hour window.

Lack of Access to Social Media Data

The original project specification envisioned incorporating social media data, particularly from platforms like Twitter/X, Instagram, and TripAdvisor, to capture real-time visitor sentiment and user-generated content. In practice, access to these data sources proved infeasible within the project's timeline and budget. Twitter/X's API pricing (introduced in 2023) made automated collection prohibitively expensive for a public-sector organization. Instagram's API restricts access to business accounts and does not support the kind of broad location-based search required for destination monitoring. TripAdvisor's terms of service explicitly prohibit automated scraping. These access restrictions meant that the pipeline's coverage is limited to traditional media sources, excluding the rich but commercially gated world of social media and review platforms.

Data Quality and Completeness Issues

Even for freely accessible articles, data quality varied significantly across sources. Some sitemaps contained stale or duplicate entries. Date formats differed across publishers, requiring a robust multi-format parser. Some articles that appeared in sitemaps returned 404 errors when scraped (having been deleted or moved between publication and collection). Google News redirect URLs occasionally fail to decode because Google changes their encryption from time to time, resulting in lost articles. These quality issues, while individually minor, collectively

required extensive defensive coding, null checks, fallback defaults, retry logic, and detailed logging, that added considerable complexity to the codebase.

Etc.

Stage 2: Relevance Analysis

URL Relevance Filtering — The Real Challenge

Filtering URLs turned out to be one of the hardest problems in the entire pipeline, and it took several failed strategies before landing on something that actually worked.

The difficulty starts at the source level. RSS feeds, which are the primary discovery mechanism, return only URLs, and URLs contain only the article's title, not its content. So, the pipeline is making relevance decisions based on a handful of words in a URL path, with no access to the body of the article. That constraint shaped everything that followed.

The Keyword Trap

Keyword filtering was the first approach tried, and it failed quickly. Searching for "québec" in URLs returned everything from Hydro-Québec press releases to provincial government announcements — the word is simply too broad to be useful. There's no way to distinguish "Québec City" from "Québec the province" through string matching alone. Adding "tourisme" as a second keyword helped marginally but introduced the opposite problem: plenty of articles deeply relevant to the tourism economy — a new Air Transat route, a hotel labour dispute, a restaurant opening — never use the word "tourism" at all.

A multi-step zero-shot classifier was also tested as a middle ground: first filter for articles mentioning Québec City specifically, then apply a second pass filtering for tourism relevance. This pipeline was meaningfully better than raw keyword matching, but still not reliable enough to trust at scale. Too many genuinely relevant articles were lost in the first filtering step before the tourism classifier even saw them, and the ambiguity problem never

fully went away. It became clear that no classification approach working only from titles and URLs — without access to actual article content — was going to be precise enough.

The Case for a Smarter Relevance Model

The idea of using a cross-encoder reranker emerged from a separate context: building a RAG (Retrieval-Augmented Generation) system for internal use. In RAG pipelines, the standard approach is to retrieve candidate documents using cosine similarity on embeddings — comparing a query vector to document vectors and ranking by distance. But cosine similarity has a fundamental weakness: it compares vectors independently, without ever considering the relationship between the query and the document together. Two texts can be close in embedding space because they share vocabulary or domain, not because one actually answers the other.

Cross-encoders solve this differently. Instead of embedding query and document separately and comparing them, a cross-encoder takes both as a single input and computes a relevance score that reflects their interaction directly. It reads them together, which means it can catch nuances — like a document that uses the right words in the wrong context, or a document that's clearly relevant even though it uses different vocabulary than the query. For ranking tasks, cross-encoders consistently outperform cosine similarity in precision.

The idea was to apply this to the URL filtering problem: use a reranker to score each article title against queries like "Québec City tourism" and filter out low-scoring ones before passing the remainder to the LLM (which I finally ended up using). This would reduce the volume hitting the LLM and hopefully catch the most obvious non-relevant content cheaply, because the caveat with LLM is that they are powerful but can be very costly depending on the volume of data input/output.

Why Reranking Didn't Save the Day

Several reranking models were tested in the hope that a fast, lightweight scoring step could pre-filter articles before they reached the LLM. The results were discouraging across the board, though for different reasons.

Jina Reranker v3 was the most promising. It produced directionally correct scores and handled French text well thanks to its multilingual training. The issue was that its raw outputs are logits with no built-in normalization, so the numbers themselves are meaningless without post-processing. Temperature-scaled sigmoid helped make the scores more readable, but choosing the right temperature was pure guesswork — and even with well-distributed scores, drawing a hard classification threshold remained arbitrary. An article scoring 0.4 might be genuinely irrelevant, or it might be a highly relevant article written in a way that doesn't surface obvious tourism signals. There was no principled way to tell.

Zerank-1-Small crashed on GPU memory. Even though I tried 4 bit quantization to reduce model memory usage. This is a structural limitation of cross-encoders more broadly: because they process the query and the full document together in a single forward pass rather than encoding them separately, memory consumption scales with the combined input length. For full news articles, this is simply too much for a model of that size.

BGE-Reranker-v2-Gemma, which uses a 2B-parameter Gemma base that is an open-source LLM free of use locally without API, fared better on short inputs — it was used to score article titles and URL slugs against a single binary tourism query, and it worked reasonably well for that. Crucially, it also supported adding an explicit instruction alongside the query, which meaningfully improved results by giving the model clearer context about what "relevant" actually meant in this setting. However, extending it to full-article multi-category classification would have introduced the same threshold and calibration problems as the other rerankers.

Voyage AI's rerank-2.5 compressed all scores into a narrow band regardless of how the queries were phrased. This is characteristic of commercial ranking APIs optimized for relative ordering — they're calibrated to distinguish between a good result and a great result within a retrieved set, not to produce absolute scores that mean something in isolation. When every article gets a score between 0.23 and 0.49, there's no threshold that separates relevant from irrelevant.

The fundamental issue underlying all of these failures is architectural. Rerankers are trained to solve a ranking problem given a query and a set of candidate documents, order them by relevance. That objective does not require — and does not produce — scores that carry consistent absolute meaning. A score of 0.7 from a reranker means "more relevant than the other documents in this batch," not "this document is 70% likely to be about tourism." Repurposing that signal as a binary classifier requires picking a threshold that the model was never trained to respect, which is why every attempt produced either too many false positives or too many false negatives. I compared the reranker though eventually to the LLM (final method used) , it was great at filtering the noise (irrelevant articles) and seemed promising, the issue is with no clear threshold we would lose good articles as well, and recall of articles is crucial for DQC so I couldn't afford to lose good articles even if we clear 80% of the irrelevant articles for the LLM. I moved away from rerankers and kept for RAG purposes only.

Letting the LLM Handle It — With a Web Search Lifeline attached to it

The truth is that no matter how capable the underlying model, working from a URL and title alone gives any classifier too little to go on. The title of a news article is often vague, abbreviated, or written for click appeal rather than descriptive accuracy — and for a niche geographic and thematic target like Québec City tourism, even a well-written title frequently omits the specific signals that would make relevance obvious.

Large language models offered a genuine improvement here, and not just because they're larger. An LLM trained on the breadth of the web carries implicit knowledge about places, institutions, companies, and events, so even when a title doesn't explicitly mention tourism, the model can recognize that "Mont-Sainte-Anne" is a ski resort in the Québec City region, or that "Air Transat" operates routes relevant to the local tourism economy. That kind of entity-level world knowledge is something no reranker or zero-shot classifier can replicate. Gemini was the natural choice given that DQc operates within Google Cloud, where it integrates cleanly into the existing infrastructure.

Choosing the right LLM :

Each LLM has attributes: knowledge cutoff and benchmarks on reasoning as well as input/output token cost, latency etc.. I referred to a very useful website: <https://www.vellum.ai/llm-leaderboard> which gives the latest LLM leaderboard and their performance on several tasks. For us, reasoning was key although URL filtering is not considered PHD level reasoning, but the reasoning benchmark still gives good indication on model capability. I test Gemini 2 and 2.5 but got fewer relevant URL's each time, Gemini 3 Flash returned the most and results were very close with Gemini 3 pro. Gemini 3 flash is a big bump in reasoning (90% on DIAMOND test) from 2.5 (around 78%), but going to pro did not justify the 5 times increase the price. So Gemini 3 flash was the sweet spot ; prompted as a media intelligence analyst with detailed instructions about Québec City's geographic scope, what counts as tourism-relevant, and what doesn't. This approach could resolve the ambiguity that keyword matching and reranking couldn't, because it reads context rather than matching strings.

Even so, the LLM had its own failure mode. When an article title is short or ambiguous, the model simply doesn't have enough information to make a confident judgment. It would sometimes hallucinate relevance, guessing that an article was tourism-related based on a vague title, or miss genuinely relevant articles because the title gave no clear signal. Obituaries got classified as relevant because they mentioned Québec City locations. Provincial policy articles got flagged as tourism content because they mentioned the word.

Even so, the LLM had its own failure mode. When a title was genuinely ambiguous or unrepresentative of the article's actual content, the model had no real basis for a decision and would sometimes assume a hallucinated relevance judgment rather than admit uncertainty. The obvious fix?: scraping the full article content before scoring, which was ruled out early. With relevant articles representing only around 5% of the total volume, fetching the full text of every URL would be prohibitively expensive and slow, most of it wasted on content that was never going to be relevant anyway.

The solution was to make the process agentic by attaching a web search tool available on the cloud to LLM. Rather than forcing the model to commit to a score on every article, it was given the ability to recognize its own uncertainty and act on it, triggering a targeted web search when confidence was low, retrieving a snippet of the actual article content, and revising its judgment with that additional context. I put 0.8 out of 1 as a confidence threshold to trigger a search with Google Custom search json API which is available on google cloud and parses google rss and can access the article snippet (first 200 ~300 characters). This preserved the efficiency of title-only scoring for the clear cases while giving the model a way to resolve the ambiguous ones without scraping everything upfront.

I ran multiple tests with this agentic framework, the LLM tended to do a search on 30% of all URL's (when LLM confidence was below 80%) which is significantly lower than analyzing all URL's and help reduce the cost of the API (approx. 7\$ per 1000 requests) by 70%.

```

print(f'{"-"*80}\n')

# Top 10 most relevant
print("TOP 10 MOST RELEVANT:\n")
for i, a in enumerate(sorted(analyzed, key=lambda x: x['relevance_score'], reverse=True)[:10], 1):
    print(f"{i}. [{a['relevance_score']}%] {a['title']}")
    print(f"    {a['reasoning']}\n")

[1]

...

[774/779] Les enfants d'un meurtrier refusent qu'il soit libéré plus t...
Score: 0% | Confidence: 1.00 - NOT RELEVANT

[775/779] SUSE et Evroc unissent leurs forces pour accélérer la souver...
Score: 0% | Confidence: 0.00 - NOT RELEVANT

[776/779] Dans le pire des scénarios, l'Outaouais perdrait 95 médecins...
Score: 0% | Confidence: 1.00 - NOT RELEVANT

[777/779] La caricature de Côté...
Score: 0% | Confidence: 1.00 - NOT RELEVANT

[778/779] DÉKUPLE DÉVOILE SON PLAN STRATÉGIQUE AMBITION 2030 ET OUVRE ...
Score: 10% | Confidence: 0.50 - LOW RELEVANCE

[779/779] CHW - Journal Le Carrefour de Québec...
Score: 0% | Confidence: 0.00 - NOT RELEVANT

=====
COMPLETE: 49/779 relevant to Quebec City (≥50%)
TOTAL GOOGLE SEARCHES: 273
Saved to: analyzed_20260114_220033.csv
=====

TOP 10 MOST RELEVANT:

1. [95%] 2025: une année touristique record à Québec
The article discusses the tourism outlook for Quebec City in 2025, indicating a record year for tourism, which directly impacts local businesses and events in the region.

```

System Prompt Design

The relevance analysis relies on a meticulously crafted system prompt of approximately 3,000 tokens that positions Gemini as an “elite Media Intelligence Analyst.” The prompt encodes five components:

1. **Geographic Scope Definitions:** Precise enumeration of five geographic zones (the Quebec City agglomeration and the MRCs of La Jacques-Cartier, La Côte-de-Beaupré, L’Île-d’Orléans, and Portneuf) with specific municipalities and tourism assets.
2. **Classification Rubric:** Four-tier scoring—Highly Relevant (90–100) for news directly about Quebec city tourism, Moderately Relevant (80–89) for indirect news about politics, economy or non-touristic news in the strict sense but might affect Quebec city tourism in an indirect way, Low Relevance (10–79), Not Relevant (0–9)—with detailed tourism-specific examples.

3. **Contextual Analysis Instructions:** Directives to evaluate context rather than keywords (e.g., “traffic” in a suburb vs. “traffic” blocking the Quebec Bridge during tourist season).
4. **Key Indicator Extraction:** Requirements to identify 1–3 named entities justifying the classification.
5. **Output Format:** Strict JSON schema enforced through Pydantic validation.

Gemini is invoked with temperature 0.0 for deterministic output, `response_mime_type` set to `application/json`, `response_schema` set to a Pydantic model (`RelevanceResult`) to make sure the LLM responds in proper Json format and handle retries on bad outputs (like a missing character or mixed responses in French and English that gave Json format parsing errors), and `thinking_budget` set to zero for minimal latency and cost. The SDK’s parsed response pathway is used for deserialization, with a fallback to manual JSON parsing when the parsed pathway fails.

Error Handling and Retry Logic

A five-attempt retry strategy with escalating backoff handles JSON decode errors, Pydantic validation failures, and API rate limiting (HTTP 429, 500, 503, `RESOURCE_EXHAUSTED`). On the second retry, non-ASCII characters are stripped from titles to reduce encoding-related failures. If all retries fail, fallback records with zero scores and `ERROR` categories are generated to prevent data loss.

The Quota Crisis: 492 API Calls for 492 Articles

```

>score: 0% - NOT_RELEVANT

[12/991] Le Canadien Samuel Montembeault cédé au Rocket de Laval...
Score: 15% - LOW_RELEVANCE

✗ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[13/991] Guide cadeaux Pour jouer dans la neige !...
Score: 0% - ERROR

[14/991] Crise avec le Venezuela Trump annonce un blocus sur les « pé...
Score: 0% - NOT_RELEVANT

✗ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[15/991] 2025 vue par La Presse Les meilleurs moments en vidéo...
Score: 0% - ERROR

[16/991] Ce qu'il faut savoir ce mardi Vague d'influenza, Pablo Rodri...
Score: 10% - LOW_RELEVANCE

✗ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[17/991] Figure du magazine Fluide Glacial Mort du dessinateur Edika...
Score: 0% - ERROR

✗ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[18/991] Mort de Matthew Perry par overdose Un des médecins assigné à...
Score: 0% - ERROR

[19/991] Jarmo Kekalainen, nouveau dg des Sabres La priorité sera de ...
Score: 0% - NOT_RELEVANT

[20/991] Relation intime secrète avec un avocat de la défense Une pro...
Score: 10% - LOW_RELEVANCE

✗ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[21/991] Billboard Hot 100 All I Want for Christmas Is You bat un rec...
Score: 0% - ERROR

✗ ERROR: 429 Quota exceeded for aiplatform.googleapis.com/generate_content_requests_per_minute_per_project_per_base_model with base model: gemini-experimental....
[22/991] Patinage de vitesse sur longue piste Défier les années selon...
Score: 0% - ERROR

```

Figure 16: Gemini API quota exceeded error on GCP

The first implementation of Gemini relevance analysis processed articles individually, one API call per article. On a typical day with 1500 discovered articles, this meant 1500 Gemini API calls for a single pipeline run. The Google Cloud Platform’s per-minute quota for the Gemini API was quickly exceeded, producing cascading 429 (Quota Exceeded) errors that caused roughly 30–40% of articles to fail with zero scores and ERROR categories. The pipeline logs showed the devastating pattern: successful scoring of an article, then a 429 error, then another success, then two more 429 errors, creating an unpredictable patchwork of scored and unscored articles.

The solution was batch processing: grouping 25 articles into a single API call and asking Gemini to score all of them at once. This reduced the number of API calls from 1500 to 60 per run, an efficiency improvement of 96%. However, batching introduced new complexities: accuracy of the LLM dropped, and it turned out to be an indexing issue. I clearly indexed the article in each batch so not to confuse the LLM and so that every relevant analysis corresponds to the article in the same order. The number 25 was chosen as the optimal balance between API efficiency (fewer calls) and response quality (the model’s accuracy degrades with extremely large batches).

This also helped dramatically reduce the input token cost caused by the prompt being invoked for each article. This way one prompt is sent for 25 article URL's. I did cache the prompt as well as it helped reduce the input cost by 90%. I made sure to delete the cache after the LLM calls are done because Google Cloud charges for cache storage per minute usage.

Output Errors

Several output-related issues emerged during pipeline development, all stemming from the inherent unreliability of working with a probabilistic large language model. Article titles containing French special characters were corrupted in Gemini's JSON responses, with "Québec" appearing as "QuÃ©bec" and "Hébert" appearing as "HÃ%BERT" in the output spreadsheets. These encoding artifacts were traced to malformed Unicode escape sequences in Gemini's JSON output, which the standard `json.loads()` parser could not handle. The fix involved a regex-based sanitizer that detects and removes malformed `\uXXXX` escape sequences before JSON parsing, plus a retry strategy that strips all non-ASCII characters from input titles on the second attempt to reduce the likelihood of encoding-related failures.

Beyond encoding, large language models are probabilistic systems that can produce inconsistent outputs for identical inputs. Despite setting Gemini's temperature to 0.0, occasional variations in output format, JSON structure, and classification decisions were observed. The Pydantic validation layer, manual JSON parsing fallback, and multi-attempt retry logic were all implemented to address this fundamental unreliability. The system prompt itself underwent approximately fifteen major revisions during development, each addressing a specific failure mode identified through testing, such as the model returning "neutral" instead of "neutre," or wrapping JSON in markdown code fences.

The most impactful single fix on the deviation side was setting the temperature parameter to 0.0. The Gemini model's tendency to deviate from the prescribed output format, including wrapping JSON in markdown code fences, adding conversational preambles before the JSON array, or returning inconsistent field names, was significantly reduced by this configuration, which forces fully deterministic output and eliminates the sampling randomness that caused format variations. Combined with the `response_schema` parameter (which instructs the API to enforce the Pydantic model structure at the generation level), temperature 0.0 reduced output

format errors from approximately 15% of batches to almost zero. By the very end of development, I had no remaining issues and the pipeline ran smoothly with no errors.

Stage 3: Playwright Web Scraping

After articles pass through the Gemini relevance filter (Stage 2, scoring above 79.5), the pipeline moves to extracting the full article text from each web page. This stage went through several iterations before I arrived at a reliable solution.

My first approach bypassed browser automation entirely, using Trafalura's built-in download capability to fetch and extract article content in a single step. This worked for some outlets but produced catastrophic results on others. TVA Nouvelles returned connection errors, and Narcity returned login page HTML instead of article content — the scraper captured text like "Need an account? Login. Already have an account? Free. \$0. Gain access to free articles..." instead of the actual news article. These failures demonstrated that many modern news websites require JavaScript execution to render their actual content, making simple HTTP-based scraping fundamentally insufficient.

My first attempt at browser automation used Selenium WebDriver with ChromeDriver, a widely-used tool for browser automation. While Selenium successfully rendered JavaScript-heavy pages, it presented significant operational challenges: ChromeDriver version management required exact version matching between ChromeDriver and the installed Chrome browser, it had poor async support making concurrent scraping difficult since Selenium is fundamentally synchronous, and its detectable automation signatures were being identified by several outlets' anti-bot systems. After extensive testing, I decided to migrate to Playwright.

Playwright, a browser automation library maintained by Microsoft, offered everything Selenium couldn't: native async support essential for concurrent scraping, built-in browser management with no version matching required, and a more modern API. Unlike simple HTTP requests, Playwright opens a full Chromium browser, executes all the JavaScript, and renders the page exactly as a real user would see it.

Articles from SerpAPI arrive as Google News redirect URLs, which need to be resolved before scraping. Resolution occurs in two phases: first, the `gnewsdecoder` library attempts parallel decoding

with concurrency limited to two and delays of 2.5 to 4.5 seconds; failed URLs fall back to Playwright-based resolution, opening the Google News page and waiting for the JavaScript redirect to complete.

Each request creates a fresh browser context with a randomized User-Agent from five Chrome, Edge, and Firefox signatures, randomized viewport sizes (1440x900 or 1920x1080), locale set to fr-CA for French domains and en-US otherwise, and matching sec-ch-ua headers. Resource blocking is applied to images and media to speed up page loads.

Even after adding browser-like headers and stealth configuration, several outlets deployed Web Application Firewalls that returned CAPTCHA challenge pages instead of article content. The scraper would receive an HTML page titled "Human Verification" containing AWS WAF JavaScript challenges — completely useless for content extraction. This failure was particularly difficult to diagnose because the HTTP response returned a 200 status code, so the scraper believed it had received the article page when in fact it had received a CAPTCHA wall. I had to add a quality check that inspects the extracted text for CAPTCHA-related keywords to detect this failure mode.

```

WARNING:trafilatura.sitemaps:invalid URL: <!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Human Verification</title>
<style>
body {
font-family: "Arial";
}
</style>
<script type="text/javascript">
window._awafCookieDomainList = [];
window._gokuProps = {
"key": "AQIDwHjcyu/GjXQlghicBg/7bfAQZ+m5FKCNDvOvTbNg9GAGCcoufQzmcBz9Xjz3nWfAAAAFJB88gkqk1G0w8BwagzB1AgFAWgGCSqGSIb3DQEHAQAQAgB1ghkgBzQMEAS4wEQQ9wY1Kv7+F6vXwKwAgQgDtclt4pP7LDfJDlyB11aumaBBo70vFvJ8ap0
"iv": "CpAHwZlvXGAPfml",
"context": "1x1g5G0k8kz2M3MEZ3/pJks6C6SRNBGLPUG5En931f0ZEP4elUAAaoIas8FjWGAZ1f5GP8ruhuadocDpYdNYF5vzAb9NC1A1FaGARQqYvVlJNSB9qD1rFmlrTzqsCRHwGKXQq4gAji6yXqQHR+6PpbwQ1fhkzrFDohxaJ4KCDsH4IsnZ7NlRg
};
</script>
<script src="https://82276e5b1049_e43c4bc8-us-east-1-token.amazonaws.com/82276e5b1049/827d4b31b10_115d4d74a9df/challenge.js"></script>
<script src="https://82276e5b1049_e43c4bc8-us-east-1-captcha.amazonaws.com/82276e5b1049/9d74de33b19c/15d4d74a9df/captcha.js"></script>
</head>
<body>
<div id="captcha-container"></div>
<script type="text/javascript">
_awsWafIntegration.saveReferer();
window.addEventListener("load", function() {
const container = document.querySelector("#captcha-container");
captchaScript.renderCaptcha(container, async (voucher) => {
await ChallengeScript.submitCaptcha(voucher);
window.location.reload(true);
});
});
</script>
<noscript>
<div>JavaScript is disabled</div>
In order to continue, you need to verify that you're not a robot by solving a CAPTCHA puzzle.
The CAPTCHA puzzle requires JavaScript. Enable JavaScript and then reload the page.
</noscript>
</body>
</html>

```

Figure 7: CAPTCHA challenge page encountered during scraping

One particularly insidious problem emerged only after the pipeline had been running for several days. Reviewing the scraped content in the BigQuery table, I discovered that several articles contained cookie consent banner text mixed into the article content. For example, one article from Ville de Québec's website had its content prepended with "Ce site utilise des témoins de navigation (cookies) essentiels pour son bon fonctionnement..." This happened because the cookie consent overlay was part of the page DOM when Trafilatura extracted the content. The solution was to implement an

automated cookie consent handler that detects and clicks accept/agree buttons before content extraction, using role-based button detection with regex pattern matching in both English and French, with CSS selector fallbacks for common consent banner implementations.

[illegible]

Figure 20: Cookie consent banner polluting scraped content

Once the page was properly rendered and cleared of overlays, the HTML is passed to Trafilatūra (Barbarese, 2021) for content extraction, configured to include tables but exclude comments.

Trafilatura is a library specifically designed for isolating main article text from navigation, advertising, sidebars, and boilerplate. It consistently outperforms more general alternatives like newspaper3k and readability-lxml on content extraction benchmarks, and its strong support for multilingual content made it the natural choice for the bilingual Quebec media landscape. Articles yielding more than 100 characters of text are marked as successfully scraped.

Concurrency is controlled by an async Semaphore with a limit of five simultaneous scraping tasks. Per-domain rate limiting ensures 6.8 to 13.6 seconds between requests to the same domain, with exponential backoff and jitter applied for HTTP 429 and 503 responses, respecting the Retry-After header when present... A blocklist of domains known to produce irrelevant content or aggressive anti-scraping responses was also maintained and updated throughout the project.

Several of the most editorially important media outlets in Quebec operate behind soft or hard paywalls. Le Devoir limits free access to a small number of articles per month, while La Presse uses a metered access model. The pipeline captures whatever content is available in the freely rendered HTML, but paywalled paragraphs that require authentication remain inaccessible. This constraint meant that the pipeline could not utilize as much data as originally envisioned, particularly from the highest-quality editorial sources most relevant to DQC's intelligence needs.

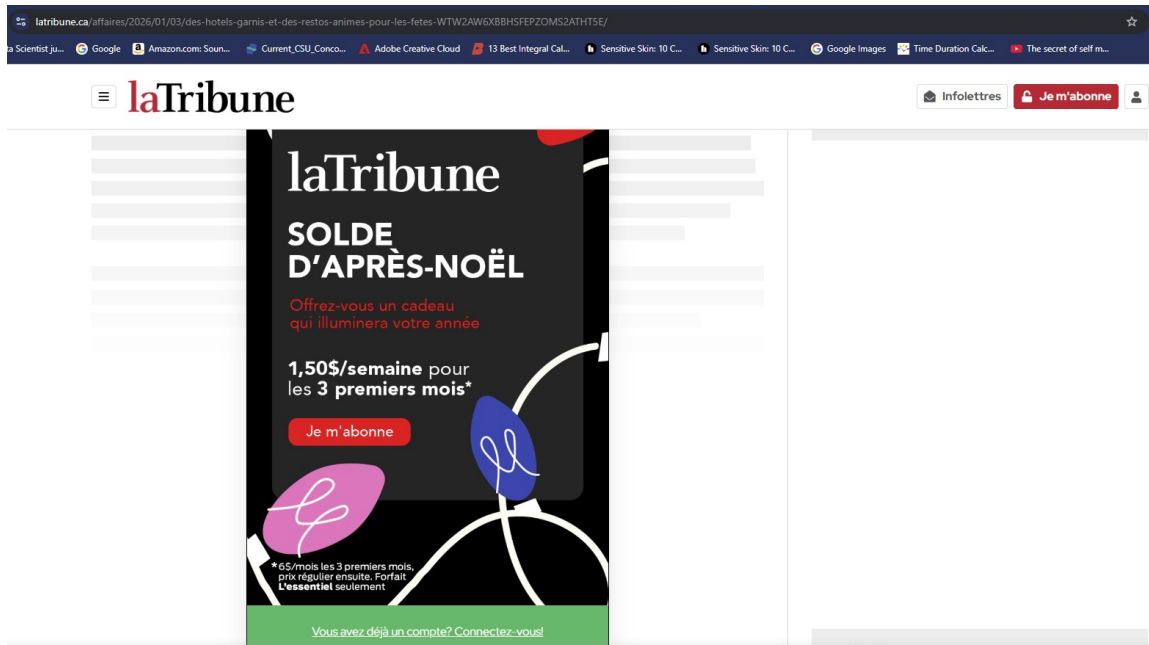


Figure 21: Paywall restriction on La Tribune requiring payment

Importantly, a deliberate decision was made not to attempt to circumvent paywalls. While techniques exist for bypassing metered access — cached versions, archive.org lookups, session manipulation — deploying such techniques in a production system operated by a public-sector organization would raise serious legal and reputational risks. The pipeline's design reflects a principled commitment to accessing only freely available content, even at the cost of incomplete coverage. This ethical boundary was discussed and endorsed by DQC's management team, who agreed that the reputational risk of paywall circumvention far outweighed the intelligence gain.

Stage 4: Sentiment Analysis

Assessing the sentiment of a full news article is harder than it sounds. A single article can contain positive facts, negative quotes, neutral background information, and mixed opinions all in the same piece. A story about a new hotel opening might praise the architecture but criticize the impact on local traffic, so how do you assign a single sentiment to that? This was a question I had to think carefully about before choosing an approach that can fit into DQC dashboard as a single word or number.

From Bag-of-Words to Deep Learning

My first instinct was to use a vocabulary-based approach. The idea is simple: build a dictionary of words where each word has a pre-assigned sentiment score, then count up the positive and negative words in a text to get an overall score. This is the logic behind well-known tools like VADER (Valence Aware Dictionary and sEntiment Reasoner) and the AFINN lexicon. I experimented with this approach early on, and while it was easy to understand and fast to run, it had serious limitations. It couldn't handle negation well ("not bad" would score as negative because of "bad"), it missed context entirely, and it struggled with French text since most sentiment dictionaries are English-focused. For a bilingual pipeline processing hundreds of articles daily, this approach was too fragile and too slow to adapt.

I briefly considered a document-term matrix (DTM) approach, where you represent articles as numerical vectors based on word frequencies and then train a classifier on top. This is a step up from raw dictionary lookup, but it still treats words as independent units and loses all contextual meaning. A phrase like "the hotel was not disappointing" would be treated the same as "the hotel was disappointing" because the same words appear.

That's what pushed me toward deep learning and transformer-based models. These models understand context, handle multiple languages natively, and have been pre-trained on massive amounts of text. The question was: which model?

Testing Models Without Data

At the start of the project, I had no scraped data yet, or very few, the pipeline hadn't been built. So I needed an external dataset to benchmark different models. I found the mx-phd/tourism sentiment dataset on Hugging Face, a multilingual (French, English, and Spanish) dataset of tourism-related tweets manually annotated at the text level with sentiment labels and at the token level with locations and thematic concepts linked to the World Tourism Organization Thesaurus of Tourism and Leisure Activities, covering the Pyrenees/Basque Country cross-border region between France and Spain, with 1,662 training samples, 619 validation samples, and 680 test samples. It wasn't an extremely perfect match

for formal news text, but it gave me a starting point to compare models in a tourism context. I tested six multilingual sentiment models .

1. `cardiffnlp/twitter-xlm-roberta-base-sentiment` — based on XLM-RoBERTa, trained on multilingual tweets (Barbieri et al., 2022; Conneau et al., 2020)
2. `nlptown/bert-base-multilingual-uncased-sentiment` — BERT-based, trained on product reviews with a 1-to-5 star scale
3. `tabularisai/multilingual-sentiment-analysis` — a multilingual model optimized for general sentiment
4. `cardiffnlp/twitter-xlm-roberta-base-sentiment-multilingual` — an extended multilingual variant of the CardiffNLP model
5. `clapAI/roberta-base-multilingual-sentiment` — RoBERTa-based multilingual sentiment classifier
6. `joeddav/xlm-roberta-large-xnli` — a large XLM-RoBERTa model used in zero-shot classification mode

I had 95.1% accuracy off the bat with Twitter-XLM-Roberta (see figure I nannexe) which was a good sign. I needed to test to see if tourism dataset is generic or too niche like the finance domain for example where you need more specialized models (like finBERT for example trained on finance dataset), it turns out tourism is mostly common and not too niche and sophisticated data.

Stress-Testing with Synthetic Data

The tourism dataset was useful for a first pass, but it contained mostly straightforward, easy-to-classify examples. I wanted to know how these models would handle harder cases ,sarcasm, mixed sentiment, negation, and implicit opinions , which are common in real-world social media news and editorial writing. Since no such dataset existed for bilingual tourism content, I used an LLM to generate a synthetic challenge set of 100 carefully crafted

examples in both English and French. Each sample was tagged with a difficulty level: medium, hard, sarcasm, mixed sentiment, tricky negation, false positive, and so on.

```
print(f"True: {true} -> Pred: {pred} (conf: {conf:.2f})")
print("-" * 90)
```

CHALLENGE LEADERBOARD - 100 Tourism Samples (EN+FR)								
Model	Acc	F1	EN	FR	Sarc	Medium	Hard	⌚
ClapAI Multilingual	55.0%	53.8%	60.0%	50.0%	38.9%	55.6%	54.7%	21s
Tabularisai Multilingual	54.0%	52.2%	52.0%	56.0%	33.3%	80.6%	39.1%	3s
Twitter XLM-RoBERTa	49.0%	49.4%	56.0%	42.0%	16.7%	69.4%	37.5%	5s
CardiffNLP XLM-R Multilingual	49.0%	48.1%	50.0%	48.0%	16.7%	58.3%	43.8%	9s
NLPtown BERT (1-5 stars)	42.0%	42.7%	40.0%	44.0%	22.2%	41.7%	42.2%	5s

The results on this challenge set were revealing. Performance dropped significantly across all models compared to the standard tourism dataset. The leaderboard on the synthetic set was:

The degradation in performance was expected; the synthetic set was deliberately adversarial. Sarcasm accuracy was especially poor across the board. French sarcastic examples like "Formidable expérience si vous aimez les cafards et les serviettes humides. Cinq étoiles pour les cafards" were confidently classified as positive, which showed a clear limitation of all models tested. This confirmed that no off-the-shelf model would perfectly handle every edge case, and that the choice of model had to be made with the actual use case in mind.

The Decision: Fine-Tune or Not?

The question of whether to fine-tune a model on domain-specific data could only be answered once I had real data flowing through the pipeline. I waited until I had collected roughly a month's worth of scraped news articles and then manually reviewed a sample of 185 articles to assess the tone and style of the content the pipeline was actually processing.

The key finding was that Quebec news articles are overwhelmingly formal in register. Unlike tweets, reviews, or social media posts, which are casual, emotional, and full of slang and sarcasm, news articles use structured, professional language with clear attribution and factual

phrasing. This formality actually works in favor of pre-trained sentiment models, because the language is less ambiguous and the sentiment signals are more explicit.

When I evaluated `cardiffnlp/twitter-xlm-roberta-base-sentiment` against this real-world news sample, performance was strong: 88.2% accuracy with a macro F1 of 0.878. The model handled formal news language well, correctly identifying sentiment in straightforward reporting and editorial content. The main confusion areas were between neutral and mildly positive articles, which is a genuinely ambiguous boundary even for human annotators.

Based on these results, I decided not to fine-tune. The pre-trained model performed well enough on formal news text that the engineering effort and ongoing maintenance burden of fine-tuning would not be justified. Fine-tuning would also introduce a risk of overfitting to the specific articles available at that time, potentially reducing generalization as the media landscape evolved.

Final Implementation

The chosen model, `cardiffnlp/twitter-xlm-roberta-base-sentiment`, is a multilingual transformer based on XLM-RoBERTa (Conneau et al., 2020), trained on the TweetEval benchmark (Barbieri et al., 2022). It is loaded from local files in the production pipeline to eliminate runtime network dependencies.

To handle the nuance problem, where a full article contains mixed signals, each article is decomposed into sentence-level phrases (minimum 20 characters, maximum 350, split on sentence-ending punctuation using a regex pattern). Each phrase is classified individually in batches of 16, and the article-level sentiment score is computed as:

$$\text{Sentiment} = (\text{Positive} - \text{Negative}) / (\text{Positive} + \text{Negative})$$

This yields a score in the range $[-1, +1]$, where neutral phrases are excluded from the denominator to focus on opinionated content. This sentence-level decomposition approach means that a mixed article naturally produces a score near zero, while a strongly positive or negative article pushes toward the extremes.

I used this approach to not deal with LLM that usually gives a single, non-nuanced, sentiment value to an article and cost more than free pre-trained model. Although this approach was effective at classifying generally bad news from good news, it was still an objective sentiment. DQC supervisor noticed some articles were bad in tone but not necessarily bad as from DQC or Quebec City point of view. An example was an article about their CEO resigning, while the news was mostly negative in tone which xlm Roberta captured (example a sad day that CEO was resigning etc...) , the news did not portray the CEO as a bad person but rather about the sad news he is leaving. So, I eventually added (in the summarization step, to avoid increasing input tokens and API calls) a “subjective” sentiment prompt to Gemini to analyze from the point of view how the image of Quebec city and its entities are portrayed in article. Training a BERT model for that seemed too hard as it requires a large amount of training data on Quebec City.

Stage 5: Semantic Article Clustering

Once all articles have been analyzed for sentiment, topic, and classification, the next step is figuring out which articles are talking about the same story. If five different outlets all publish articles about a new hotel opening in Old Quebec, I want those grouped together so DQC can see the full picture of how a single story is being covered across the media landscape rather than looking at five separate entries that are really about the same thing.

Even though we did deduplicate earlier by same URL and same title, clustering / grouping was still necessary if different news outlets relay same story where the title doesn't necessarily match word for word, but the content was almost similar. So semantic analysis was necessary for better dashboard readability and visualization.

Choosing the Embedding Model

To group articles by similarity, I first needed a way to represent each article as a numerical vector, essentially converting text into a point in a high-dimensional space where similar articles end up close together. This is what sentence embedding models do.

Choosing the right embedding model started with the MTEB (Massive Text Embedding Benchmark) leaderboard available on Huggingface (link : <https://huggingface.co/spaces/mteb/leaderboard>), which ranks embedding models across a range of tasks like retrieval, clustering, classification, and semantic similarity. It's a useful starting point, but it's important to keep in mind that the MTEB leaderboard is based on a broad set of English-centric benchmark datasets , it gives you a general idea of performance, but it doesn't tell you how a model will perform on your specific data, especially in a bilingual French-English news context. So, I treated it as a shortlist, not as a final answer.

Still some models

From the leaderboard, I narrowed down to a few candidates. I initially tried smaller, faster models like the paraphrase-multilingual family, which are popular and lightweight and completely free. But I quickly ran into a problem: these models have a maximum context window of only 128 or 256 tokens, which is far too short for news articles. A typical Quebec news article runs anywhere from 500 to 2,000 words. With a small context window, the model would only encode the first few sentences and ignore the rest of the article, which meant two articles about completely different topics could end up with similar embeddings just because they happened to start with similar introductory paragraphs. The clustering results were poor and unreliable.

Trying to find the right balance between model size and non API costly like gemini 001 embeddings, I went towards BAAI/bge-m3, which is a battle tested widely used embedding model free to download and can run on a 16GB CPU (which I ended up using in the Cloud deployment) , quite fast and has a context window of 8,192 tokens , large enough to encompass the full text of virtually every article in the pipeline. It also handles multilingual content natively, produces 1,024-dimensional embeddings, and ranked highly on the MTEB leaderboard for both retrieval and clustering tasks. I validated the model's performance manually over the course of about two months, regularly reviewing the clusters it produced to make sure articles that should be grouped together actually were, and that unrelated articles weren't being lumped in by mistake. Over time, I became confident that bge-m3 was producing meaningful, reliable groupings for this specific use case.

Why Graph-Based Clustering Instead of a Traditional Algorithm

Once every article is encoded into a vector, the natural next step is to cluster them. The typical approach would be to use a clustering algorithm like K-means, DBSCAN, or hierarchical agglomerative clustering. I considered these but decided against them for practical reasons.

K-means requires you to specify the number of clusters in advance, which is impossible in this context, I have no idea how many distinct news stories will emerge on any given day. It could be twelve, it could be forty. DBSCAN handles this better since it discovers clusters automatically, but it relies on density parameters (epsilon and minimum samples) that are sensitive to the data distribution and would need constant tuning as the volume and diversity of articles changed over time. Hierarchical clustering HDBSCAN (which I used earlier when doing BertTopic) has too many hyperparameters which were hard to optimize to get converge to a high coherence score, since we did not have too many data points, and the majority were unique articles as opposed to duplicate articles , so most articles turned out to be outliers affecting coherence score or making it not too representative. I had very little data in the beginning when doing BertTopic inspired by article : , which made it harder for optimization purposes. I tried Optuna for hyperparameter optimization with random search and Bayesian search but couldn't get high coherence scores with little data points. That being sad, it wasn't a bad thing after all because the goal here was to assess similarity and not group by theme. Even if 2 articles tackle the same actuality but angle in which the news outlet handle the news is in a different direction or more deep , I don't want these 2 articles to be clustered together. No clustering algorithm gives you that level of semantic precision out of the box without significant tuning.

Instead, I went with a simpler and more transparent approach: threshold-based graph clustering. The logic is straightforward. After encoding all articles into vectors, I computed a pairwise cosine similarity matrix , essentially measuring how similar every article is to every other article. Any pair of articles with a cosine similarity above 0.85 is linked together as an edge in an undirected graph. Then I extract connected components from that graph using

depth-first search. Each connected component becomes a cluster, a group of articles that are all sufficiently similar to at least one other article in the group.

The beauty of this approach is that it requires only one parameter: the similarity threshold. That single number (0.85) is easy to understand, easy to validate by reviewing actual article pairs, and easy to adjust if needed. It also handles the variable number of clusters naturally, if there are three big stories and twenty standalone articles on a given day, the algorithm produces three clusters and leaves the rest unclustered. Articles that don't match anything else retain a cluster ID of zero, which simply means they're standalone stories. (it turned that around +/- 80% of articles were in Cluster 0)

After the encoding is complete, the model is explicitly deleted from memory, and the GPU cache is cleared using garbage collect `gc.collect()` and `torch.cuda.empty_cache()` to free up some memory space . This is necessary because bge-m3 is a large model, and the pipeline needs to free that memory for downstream models that run in subsequent stages.

Stage 6: IPTC Topic Classification

I thought about another classification for better dashboard readability .Since every news article covers a topic , politics, economy, sports, arts, science, and so on and the media review was deliberately made to cover a large scope of news either related directly to tourism or having an indirect impact on the tourism landscape (e.g. : politics, economics etc.). To make the scraped data useful for DQc's dashboard, each article needs to be tagged with a standardized topic category so the team can filter and analyze coverage by subject area. For example, they might want to see all articles related to "arts, culture and entertainment" or track how much coverage "economy, business and finance" is getting relative to "lifestyle and leisure."

I chose to use the IPTC (International Press Telecommunications Council) taxonomy for this purpose. IPTC NewsCodes are the global standard used by news agencies like Reuters, AFP, and the Associated Press to categorize their content. Using this established standard rather than inventing custom categories means the classification is consistent, widely understood, and compatible with how the international news industry already organizes content.

The model I used is `classla/multilingual-IPTC-news-topic-classifier`, which assigns each article one of 17 standardized IPTC top-level categories including society, politics, economy/business/finance, sports, arts/culture/entertainment, science and technology, health, education, environment, crime/law/justice, disaster and accident, weather, religion, labour, human interest, lifestyle and leisure, and conflict/war/peace. The model was trained on a large corpus (like AG news, etc.) of professionally categorized news articles from multiple languages, which makes it a natural fit for this pipeline since it was specifically designed for the exact task I needed, classifying real news content, not social media posts or product reviews. It handles both French and English natively, which is critical for the bilingual Quebec media landscape.

The input text for classification is constructed by concatenating the article title with the first 400 characters of content. The model has a maximum token limit of 512, but in practice this is not a limitation. News articles follow a well-established journalistic convention called the inverted pyramid structure, where the most important information, the who, what, where, when, and why, appears in the headline and the opening paragraph. By the time you've read the title and the first few sentences, you almost always know what topic the article covers. In my experience reviewing thousands of classified articles over several months, the topic is identifiable from the first 400 characters in well over 99% of cases. Feeding the entire article would not meaningfully improve classification accuracy but would significantly slow down processing.

Articles are processed in batches of 8 for efficiency. The model outputs an IPTC category label in English, and a hardcoded French translation map provides bilingual labels, stored as `iptc_topic` and `iptc_topic_fr`, so that the downstream Looker Studio dashboard can display topics in whichever language the user prefers.

Stage 7: GLiClass Multi-Label Classification

What We Were Trying to Do

We had a specific problem: given a piece of text (a news article, a listing, a description, in either English or French), we needed to automatically tag it as being about one or more of three

categories: attraction (tourism events, museums, destinations), lodging (hotels, B&Bs, accommodations), or restaurant (dining, food establishments), which are the categories DQC told me they would most want to filter and monitor as relevant info to update their website and advise their shareholders. Some texts belong to none of these categories, and some belong to more than one. This was a multi-label classification task that needed to work in both English and French and handle genuinely difficult cases like the French word "restauration" (which means renovation, not restaurant in some cases), or "Hôtel de Ville" (which means City Hall, not a hotel), or "chambre de commerce" (chamber of commerce, not a hotel room).

Starting with Zero-Shot on Synthetic Data

Since I didn't have scraped articles yet at the beginning, I needed a way to test the model before real data was available. I started by testing the off-the-shelf GLiClass-x-base model (built on DeBERTa v2) in zero-shot mode on an LLM generated synthetic bilingual dataset of 100 EN+FR samples I created to represent the three categories. The results shown in Figure 1 were mixed. Attraction only reached 71.1% accuracy, lodging 79.5%, and restaurant performed best at 94.0%. The model was picking up on restaurant-related language fairly well but was clearly struggling with broader or more ambiguous categories like attraction.

Moving to Real Scraped Data

Once the scraping pipeline was running, I used a long-context LLM to extract paragraphs from a month's worth of scraped articles that pertained to the three categories. I then re-tested the zero-shot GLiClass model on this real DQC tourism data (Figure 2). Performance shifted: attraction hit 77.7%, lodging jumped to 89.3%, and restaurant held steady at 94.4%. While lodging and restaurant were looking more reasonable, attraction was still lagging and producing too many false negatives, meaning real attraction-related articles were being missed.

The Label Wording Problem

At this point I realized the model's performance was heavily dependent on how the labels themselves were worded. "Attraction" as a single label was too broad and vague. The model was failing to tag articles about museums, tourism events, or resorts as attractions because those words weren't in the label. I experimented with expanding the label to something more descriptive like "attraction museum touristic event resort" to give the model more signal. As

shown in Figure 3, this did push attraction accuracy up to 86.1% and restaurant to 94.7%, but it came at a cost: lodging dropped to 77.7%, likely because the longer, noisier label was creating interference across categories. It became clear that zero-shot classification with multi-word label engineering was a fragile approach. Performance was too sensitive to label phrasing, and there was no clean way to improve one category without hurting another.

The Dataset I Built for Fine-Tuning

Rather than keep fighting with label wording, I decided to fine-tune the model properly on real labeled data, using clean single-word labels. I used the scraped dataset I gathered from a month of the pipeline's scrapings consisting of 642 labeled examples in both English and French generated by a long context LLM. I made sure its only paragraphs no longer than 512 tokens to construct test set This wasn't a random scrape, every example was written deliberately to teach the model what we cared about. The dataset included easy examples for each category like straightforward sentences such as "Montmorency Falls welcomes over a million visitors each year" (attraction) or "Le Fairmont Château Frontenac a complété une rénovation de 75 millions" (lodging). It also included tricky hard examples designed to catch common mistakes, such as "La restauration des fresques de la chapelle a nécessité 2 millions de dollars" which is about renovation and should NOT be tagged as restaurant, or "Hôtel de Ville de Québec will open its doors to the public for Heritage Day" which is City Hall and should be tagged as attraction, not lodging. I also included "none" examples covering texts about politics, sports, crime, weather, business, and science ,things a tourism classifier should ignore entirely , as well as multi-label examples for texts that legitimately belong to more than one category, like a hotel that also has a famous restaurant. Each example was tagged with its language and difficulty level so I could track exactly where the model was struggling.

The Architecture: What We Actually Fine-Tuned

We didn't fine-tune the entire GLiClass model end to end. Instead, we used a cross-encoder approach built on top of GLiClass's DeBERTa backbone.

First, we extracted the DeBERTa encoder from GLiClass. GLiClass is a wrapper around a DeBERTa v2 model of about 280 million parameters, and we pulled out just the encoder , the part that reads text and turns it into numerical representations. Then we quantized it to 4-bit

precision using QLoRA's NF4 quantization, which shrinks the model's memory footprint by roughly 8x while keeping most of its intelligence. Instead of changing the original model weights directly, we attached small trainable LoRA adapter layers to the attention mechanism, specifically to the query, key, value, and dense projections inside each of the 12 DeBERTa layers, with a rank of 16 and an alpha of 32. The result was that only 2.65 million parameters were trainable out of 280 million total, less than 1% of the model. On top of the encoder we added a tiny linear classification head of just 769 parameters (768 inputs from DeBERTa's hidden size to 1 output) that takes the [CLS] token embedding and produces a single confidence score.

The cross-encoder approach meant that instead of encoding text and labels separately, we fed them together as a pair. For each article we created three separate inputs: text paired with "attraction", text paired with "lodging", and text paired with "restaurant". Each pair goes through DeBERTa as a single sequence, the [CLS] embedding comes out, the classification head produces a logit, and sigmoid turns that into a 0-to-1 probability. Above 0.5 means a match. This is why the 642 original examples became 1,347 training pairs and 579 test pairs, every text gets scored against every label independently.

How We Split the Data

We used a 70/30 train/test split with careful stratification, making sure each label and difficulty level was proportionally represented in both sets. Very rare categories were merged into a "rare_other" bucket so stratification wouldn't fail. The training pairs had a natural imbalance of 364 positive pairs versus 983 negative pairs, so we used a weighted loss function with a `pos_weight` of 2.70, which tells the model that a missed positive costs 2.7x more than a missed negative. This prevents the model from learning the lazy strategy of just saying "no" to everything.

Training Details

We used AdamW with different learning rates, $2e-4$ for the LoRA-adapted encoder and $1e-3$ for the classification head since the head starts from random weights and needs to learn faster. The loss function was Binary Cross-Entropy with Logits with the `pos_weight` described above. We trained for 5 epochs with a batch size of 8, a max sequence length of 256 tokens, a linear

warmup over the first 10% of steps followed by linear decay, and gradient clipping at a max norm of 1.0 for stability.

What Happened During Training

Epoch 1 was dramatic. The loss started high around 1.0 to 1.5 and accuracy hovered near coin-flip level for the first 80 or so batches, then around batch 100 something clicked, the loss dropped sharply and accuracy jumped toward 1.0. Validation accuracy after just epoch 1 was already 95.4%. Epochs 2 through 5 were about refinement, with validation loss dropping from 0.30 to around 0.10 and staying stable with no sign of overfitting. The training and validation curves declined in parallel and converged near the top, meaning the model generalized well rather than memorizing training data.

The Final Results

On the 193-row test set, the fine-tuned model reached 95.9% accuracy on attraction, catching 43 out of 46 actual attractions (93.5% recall) with only 5 false positives. For lodging it hit 95.9% accuracy, catching 51 out of 52 actual lodgings (98.1% recall) with 7 false positives. For restaurant it reached 95.3% accuracy, catching 52 out of 54 actual restaurants (96.3% recall) with 7 false positives. The score distributions told an even better story: negative examples clustered sharply near 0.0 and positive examples near 1.0, with almost no overlap around the 0.5 threshold. The model wasn't just barely getting things right, it was highly confident in its answers.

How It Handled the Tricky Cases

The results on deliberately hard examples were particularly satisfying. "La restauration du patrimoine bâti dans le Vieux-Québec coûtera 15 millions" received a restaurant score of 0.000, the model had learned that "restauration" in French with no food context usually means renovation. "L'Hôtel du Parlement offre des visites guidées gratuites" scored 1.000 for attraction and 0.001 for lodging, correctly identifying it as parliament. "A new farm-to-table restaurant on Rue Saint-Paul" scored 1.000 for restaurant with near-zero scores for the other two. "The Fairmont Le Château Frontenac completed a \$75 million renovation" scored 1.000 for lodging and 0.000 for everything else. The zero-shot model had gotten every single one of these wrong.

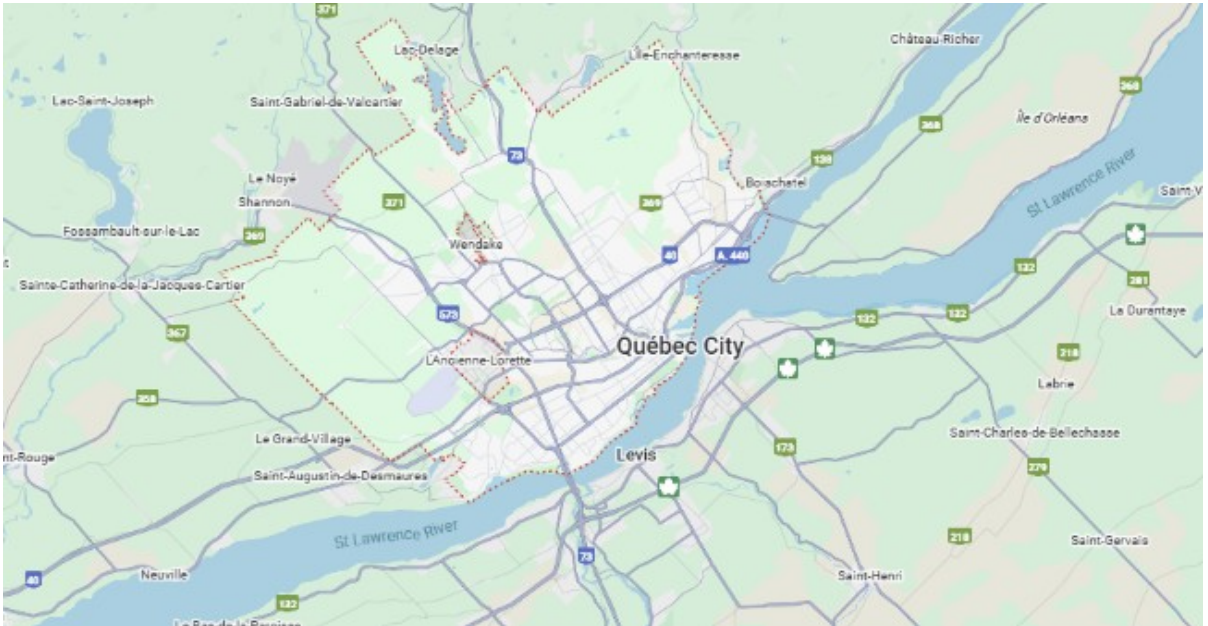
Why This Worked

A few things came together to make this successful. DeBERTa v2 is excellent at understanding text pairs since it was designed for natural language inference, and the GLiClass-x-base variant was already pre-trained on multilingual data giving it a head start on French. QLoRA let us fine-tune a 280M parameter model on a single GPU by only training 1% of the parameters. The cross-encoder formulation of text plus label as a pair is a natural fit for "does this text match this category?" questions. The 642 carefully crafted examples with intentional adversarial cases taught the model exactly where the boundaries are, and those tricky examples were critical because without them the model would have learned the same superficial word-matching patterns that fooled the zero-shot version. The jump from roughly 73% to 96% accuracy happened fast, mostly within the first epoch, because the base model already understood language well and we were just teaching it our specific decision boundaries rather than teaching it French or English from scratch. I tried to compare the zero shot model vs the fine tuned model on edge hard cases and the fine tuned model outperformed the zero shot model as you can see in image.

Stage 8: Gemini Summarization and Perception Analysis

The final analytical stage uses a second Gemini system prompt (~3,500 tokens) positioning the model as the “Senior Strategic Reputation Analyst for Destination Québec.” The prompt encodes four analytical dimensions:

- **Flag Logic:** Binary keep/discard decision (Flag 0 = keep, Flag 1 = discard). Default is to keep (Flag 0); discarding requires high confidence of zero connection to Quebec City tourism. Now that the full articles are scraped and fed to the LLM, this step served as a second relevance filter beyond Stage 2 in case the LLM hallucinated a false relevance score, probably because not having access to the full article content in stage 2. I took the chance to of few shots prompting for the LLM for weather news with no tourism impact and health sector news as well as other news DQC deemed not interested in, again because the LLM have access to the full article now and can make better judgements. The LLM struggled to follow my instructions with zero shot prompting but I got better results with few shots prompting especially for the weather nuance.
- **Geographic Tagging:** Binary field (quebec = oui/non) indicating whether the article specifically addresses the Capitale-Nationale region, with comprehensive geographic definitions for five zones, which will help DQC quickly filter local from regional news.



- **Perception Analysis:** Three-class assessment (positive/negative/neutre) evaluating impact on visitor perception and investor confidence. This is the subjective sentiment analysis added upon the objective article sentiment score in stage 4
- **Abstractive Summarization:** 1–2 sentence summary in the article’s original language, strictly grounded in the text without inference. I limited the size of summarization for quick scrolling and reducing very expensive output cost (more than 5 times pricier than input tokens) of the Gemini 3 flash LLM.

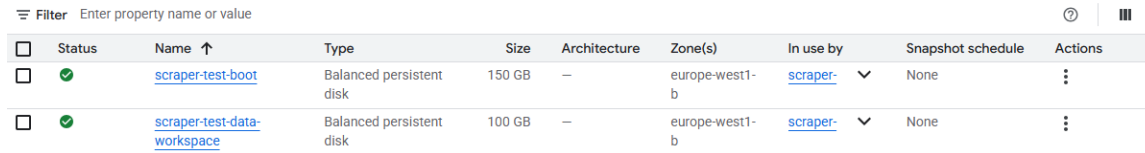
Deployment and Cost monitoring

Output is validated by a Pydantic model (ArticleAnalysis) with field-level validators normalizing linguistic variants (e.g., “négative” → “negative”) which the LLM used to wrongly output because articles are multilingual, (even though I set temperature to 0). These issues caused the output to fail at loading with proper Json format so Pydantic was necessary as an output validator. Articles are processed sequentially with 2-second delays between API

calls. Each receives up to five retry attempts with escalating backoff and content truncation on failure.

The Gemini API calls in Stages 2 and 8 represent the primary ongoing cost of operating the pipeline. Each daily run makes approximately 1000 to 1500 URL's relevance analysis and 30 to 120 individual summarization calls. Cost management strategies include batch processing in Stage 2 (25 articles per call rather than individual calls), content truncation (capping article text at 9,000 characters for summarization), and the relevance threshold filter (scraping and analyzing only articles scoring 80+, which reduces the volume reaching the expensive summarization stage by approximately 60–80%).

Idle Storage Costs



The screenshot shows a table of persistent disks in the GCP console. The table has columns for Status, Name, Type, Size, Architecture, Zone(s), In use by, Snapshot schedule, and Actions. Two disks are listed: 'scraper-test-boot' (150 GB) and 'scraper-test-data-workspace' (100 GB), both of type 'Balanced persistent disk' in the 'europe-west1-b' zone, and both are in use by 'scraper-'. Both have a 'None' snapshot schedule.

Filter	Enter property name or value								
☐	Status	Name ↑	Type	Size	Architecture	Zone(s)	In use by	Snapshot schedule	Actions
☐	✓	scraper-test-boot	Balanced persistent disk	150 GB	—	europe-west1-b	scraper-	None	⋮
☐	✓	scraper-test-data-workspace	Balanced persistent disk	100 GB	—	europe-west1-b	scraper-	None	⋮

Figure 15: GCP idle storage cost problem

Early in the cloud deployment phase, the project used persistent VM instances with attached storage disks for development and testing. Two balanced persistent disks totaling 250 GB (a 150 GB boot disk and a 100 GB data workspace) were provisioned in europe-west1-b. These disks continued to incur storage charges even when the VM instances were stopped, resulting in unexpected GCP billing. The problem was identified during a cost audit and resolved by deleting the persistent disks, migrating the pipeline to Cloud Run Jobs (which uses ephemeral storage that exists only during execution), and implementing a strict policy of cleaning up temporary cloud resources after each development iteration.

GPU Instance Stockouts

The pipeline's transformer models (sentiment analysis, embeddings, IPTC classification, GLiClass) benefit significantly from GPU acceleration. However, GPU-equipped Cloud Run

instances were frequently unavailable due to capacity constraints in the selected GCP zones. Attempts to provision GPU instances in us-central1-a returned “STOCKOUT” errors with the message “the zone does not have enough resources available to fulfill the request.” This meant that the pipeline could not rely on GPU availability for production execution. The solution was twofold: first, the pipeline code was written with automatic CPU fallback (`torch.cuda.is_available()` checks at every model loading point), ensuring it runs correctly without GPU; second, the deployment configuration was adjusted to request GPU instances but gracefully degrade to CPU instances when GPUs are unavailable, accepting slower execution (roughly 2x) in exchange for guaranteed availability.

Stage 9: BigQuery Data Persistence

The final stage writes the DataFrame to Google BigQuery with a 21-column schema capturing the complete analytical provenance of each article:

Column	Type	Description
url	STRING (REQUIRED)	Resolved publisher URL
title	STRING	Article headline
source	STRING	Publisher domain
date	STRING	Publication timestamp
relevance_score	INTEGER	Gemini score (0–100)
category	STRING	Relevance tier
reasoning	STRING	10-word justification
key_indicators	STRING	Named entities
scraped_content	STRING	Full article text
scrape_status	STRING	Scraping outcome
sentiment	FLOAT64	Sentiment (–1 to +1)
cluster_id	INTEGER	Cluster identifier

iptc_topic / iptc_topic_fr	STRING	Topic label (EN/FR)
Attrait	FLOAT64	Attraction relevance (0–1)
Hebergement	FLOAT64	Lodging relevance (0–1)
Restaurant	FLOAT64	Restaurant relevance (0–1)
quebec	STRING	Geographic tag (oui/non)
perception	STRING	Visitor perception
summary	STRING	1–2 sentence summary
processed_timestamp	TIMESTAMP	Pipeline execution time

As DQC had other projects they are currently using like time series analysis and so on , being to able to relate positive or negative signals to news data was crucial for DQC . So I made sure data is appended using WRITE_APPEND disposition, preserving historical records. A final URL-based deduplication pass removes duplicates before writing using SQL query. The pipeline includes self-provisioning logic: if the dataset or table does not exist, they are created automatically. Also a scheduled SQL query was made to delete data from Bigquery (for storage cost purposes) after being saved on an external downloadable file for data older than 1 year.

Managing Historical Data

```

SQL

CREATE OR REPLACE VIEW `clean-node-480723-k8.quebec_news.view_latest_articles` AS
SELECT *
FROM `clean-node-480723-k8.quebec_news.analyzed_articles`
WHERE processed_timestamp = (
  SELECT MAX(processed_timestamp)
  FROM `clean-node-480723-k8.quebec_news.analyzed_articles`
);

```

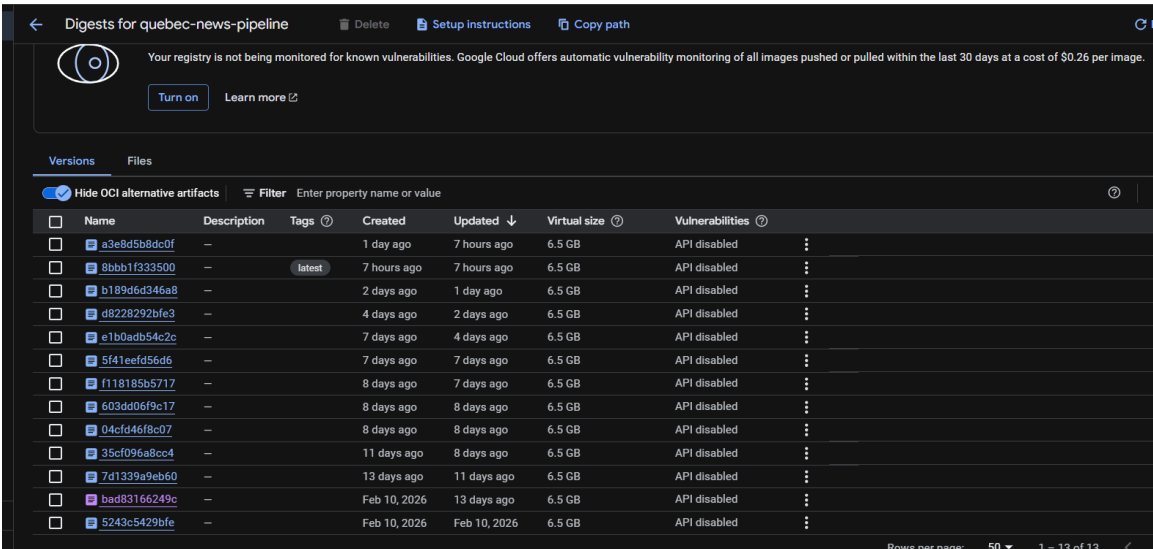
Figure 19: BigQuery historical data management view

Because the pipeline appends new records on every daily run, the BigQuery table grows continuously. To serve the dashboard's need for "today's articles only," a SQL view was created that selects only the most recent batch of processed articles. This view filters on the maximum processed_timestamp, ensuring the dashboard always reflects the latest pipeline run while preserving the full historical archive in the underlying table for trend analysis and audit purposes. This approach was not part of the original design but emerged as a practical necessity once the pipeline had been running for several days and the dashboard became cluttered with accumulated data from previous runs.

Chapter 2: Results and Analysis

This chapter presents the results of the deployed pipeline, including general performance metrics, qualitative assessment of output quality, and—critically—the feedback received from DQc’s management team and analysts regarding the system’s operational value. Rather than a traditional statistical analysis (which would be more appropriate for a research-oriented project), the evaluation here reflects the applied, operational nature of the mandate: the primary success criterion is whether the system delivers value to its intended users.

Pipeline Performance Overview



← Digests for quebec-news-pipeline Delete Setup instructions Copy path

Your registry is not being monitored for known vulnerabilities. Google Cloud offers automatic vulnerability monitoring of all images pushed or pulled within the last 30 days at a cost of \$0.25 per image.

Turn on Learn more

Versions Files

Hide OCI alternative artifacts Filter Enter property name or value

☐	Name	Description	Tags	Created	Updated	Virtual size	Vulnerabilities
☐	a3e8d5b8dc0f	—		1 day ago	7 hours ago	6.5 GB	API disabled
☐	8bbb1f333500	—	latest	7 hours ago	7 hours ago	6.5 GB	API disabled
☐	b189d6d346a8	—		2 days ago	1 day ago	6.5 GB	API disabled
☐	d8228292bfe3	—		4 days ago	2 days ago	6.5 GB	API disabled
☐	e1b0adb54c2c	—		7 days ago	4 days ago	6.5 GB	API disabled
☐	5f41eefd56d6	—		7 days ago	7 days ago	6.5 GB	API disabled
☐	f118185b5717	—		8 days ago	7 days ago	6.5 GB	API disabled
☐	603dd06f9c17	—		8 days ago	8 days ago	6.5 GB	API disabled
☐	04cfd46f8c07	—		8 days ago	8 days ago	6.5 GB	API disabled
☐	35cf096a8cc4	—		11 days ago	8 days ago	6.5 GB	API disabled
☐	7d1339a9eb60	—		13 days ago	11 days ago	6.5 GB	API disabled
☐	bad83166249c	—		Feb 10, 2026	13 days ago	6.5 GB	API disabled
☐	5243c5429bfe	—		Feb 10, 2026	Feb 10, 2026	6.5 GB	API disabled

Rows per page: 50 1 - 13 of 13

Figure 44: Cloud Run job execution dashboard

Stakeholder Feedback and Organizational Impact

The true measure of this project’s success lies not in technical metrics but in its reception by the organization. Throughout the development process, regular demonstrations and review sessions were held with DQc’s team, including the director, the internship supervisor, and the professor, as well as written feedback from analysts. Their feedback shaped the system at every stage and ultimately validated its operational utility.

DQc’s management team expressed strong satisfaction with the system’s ability to surface relevant articles that their manual monitoring process had previously missed. The perception analysis feature was described as “exactly what we needed” by the communications director,

who noted that it enabled the team to shift from reactive crisis management to proactive narrative monitoring.

The clustering feature, while less immediately visible to end users, received positive feedback from the strategy team. By grouping multiple articles covering the same event or story into a single cluster, the dashboard avoided presenting redundant information and instead surfaced the breadth of coverage, showing, for example, that a new Fairmont Frontenac renovation was covered by six different outlets rather than presenting six separate, identical summaries.

Perhaps the most telling indicator of the project's success was the behavioral change it induced within the organization. Within two weeks of the dashboard going live, DQC's morning briefing meetings began incorporating the pipeline's outputs as a standard agenda item. Analysts who had previously spent hours each morning manually scanning news websites reported that the pipeline reduced their monitoring time to approximately 15–20 minutes, with higher coverage and fewer missed articles. The time savings freed the analytical team to focus on higher-value activities: drafting strategic recommendations, preparing member communications, and developing thematic reports.

The cost of my pipeline of roughly 18\$ a month was extremely low and several order of magnitude less than what the company was paying for, in terms of enterprise packages for manual news gathering services, and with higher news recall and more in depth analysis and filtering capabilities. The pipeline was gathering between 30 and 110 articles a day (usually on weekends it is quitter news wise) , compared to 3 to 25 articles of other news services.

Chapter 3: Knowledge Transfer

A central concern for any applied project within an organization is ensuring that the knowledge, systems, and capabilities developed during the engagement survive beyond the project's formal end date. This chapter describes the knowledge transfer mechanisms put in place to ensure DQc's long-term autonomy in operating, maintaining, and extending the pipeline.

Technical Documentation

The pipeline codebase is accompanied by comprehensive technical documentation covering the system's architecture, configuration parameters, deployment procedures, and troubleshooting guides. The `main.py` source file is extensively commented, with each stage preceded by a prominent section header and configuration block. Every configurable parameter (batch sizes, thresholds, delays, model names) is externalized as a named constant at the top of the relevant code section, with inline comments explaining its purpose and acceptable value ranges. A separate README document provides step-by-step instructions for building the Docker container image, deploying to Cloud Run, configuring Cloud Scheduler, and verifying successful execution. I put the LLM model as a environment variable because older models get depreciated and constant model updating will be required by non-deevloppers , this way it assured they can change the model name outside the code without needing to reconstruct the Docker image.

Prompt Documentation and Maintenance Guide

Given the critical role of the two Gemini system prompts (relevance analysis and summarization), a dedicated prompt maintenance guide was prepared. This document explains the structure and purpose of each prompt section, provides guidelines for modifying the geographic scope definitions (e.g., if DQc's territory expands to include new municipalities), and describes the testing methodology for validating prompt changes against a reference set of manually classified articles. The guide emphasizes that prompt modifications should be tested incrementally, changing one section at a time and comparing outputs against the reference set, to avoid unintended regressions.

Dashboard and End-User Training

Two training sessions were conducted with DQC’s analytical staff, covering the BigQuery data schema, the dashboard interface, and the interpretation of pipeline outputs. Topics included understanding the meaning of each analytical field (sentiment score ranges, perception categories, cluster IDs), identifying common failure modes (partial scrapes, low-confidence classifications), and using the dashboard’s filtering and sorting capabilities to answer specific business questions (e.g., “Show me all negative-perception articles about hotels in the past week”).

Operational Handover

The pipeline was designed from the outset for autonomous operation. It runs daily on Cloud Scheduler without human intervention, handles errors gracefully through retry logic and fallback defaults, and logs all activity to Cloud Run’s integrated log explorer. Logging helped code be observable at each step and track if any new errors can occur (sometimes the websites update their RS links or example)

The nature of this project—building a production system with clear inputs, outputs, and operational procedures—makes knowledge transfer relatively straightforward. The system’s value is embodied in its daily outputs, not in tacit expertise that resides only in the developer’s head. DQC’s team can operate the dashboard, interpret the outputs, and manage the Cloud Run deployment without ongoing external support. For deeper technical modifications (adding new sitemap sources, adjusting classification models, modifying prompts), the documented codebase and maintenance guides provide a solid foundation for either internal staff or future consultants.

Proposed Ideas

This section presents two experimental approaches that were explored during the project’s development phase. While not integrated into the final production pipeline, they represent promising directions for future enhancement of the system’s analytical capabilities.

Aspect-Based Sentiment Analysis (ABSA) with Entity-Level NER

Beyond the article-level sentiment scoring implemented in Stage 4, we explored a more granular approach: Aspect-Based Sentiment Analysis (ABSA) that ties sentiment directly to specific named entities mentioned within each article. The core idea was to move from asking “is this article positive or negative?” to asking “what is the sentiment toward each specific person, organization, or place mentioned in this article?”

The approach used a two-stage pipeline. First, Named Entity Recognition (NER) was performed using the GLiNER Large v2.5 model (`urchade/gliner_large_v2.5`), a generalist NER model capable of extracting entities in both English and French without requiring domain-specific fine-tuning. GLiNER identifies persons, organizations, locations, and other entity types from raw article text, producing entity spans with confidence scores. The model’s zero-shot capabilities made it particularly suitable for our multilingual context, as it could recognize entities like “Fairmont Château Frontenac,” “Bruno Marchand” (the mayor of Quebec City), or “Destination Québec cité” without any prior training on Quebec-specific data.

Second, for each extracted entity, we identified the phrase or sentence in which it appeared (using the phrase as the context window) and ran sentiment analysis on that specific phrase using the same `cardiffnlp/twitter-xlm-roberta-base-sentiment` model employed in the main pipeline. This produced entity-level sentiment scores: for each article, we could determine not just the overall tone, but the sentiment associated with each specific entity mentioned. The results were promising. For example, in an article discussing Quebec City politics, the model correctly identified “Bruno Marchand” as a person entity with 7 mentions across the article corpus, with neutral sentiment (0.989 confidence) and entity confidence of 0.835.

Similarly, “Christian Mars” appeared 4 times with neutral sentiment, and “Stéphane Lachance” appeared 3 times. The system also tracked which specific articles each entity was mentioned in, enabling cross-article entity tracking — a capability that would allow DQc to monitor how specific stakeholders, businesses, or landmarks are being portrayed across the media landscape over time (see Figure 61).

The cluster column in the ABSA output indicated which article clusters each entity appeared in, enabling correlation between entity sentiment and story-level coverage patterns. An entity appearing across multiple clusters with shifting sentiment could signal an evolving narrative — exactly the kind of intelligence that DQc’s communications team would find actionable. However, several limitations prevented immediate production deployment. The GLiNER model’s entity extraction, while generally accurate, occasionally merged distinct entities or split compound names incorrectly in French text. The phrase-level context window approach, while computationally efficient, sometimes missed sentiment expressed in surrounding sentences that was clearly directed at the entity. A more sophisticated approach using coreference resolution and wider context windows would improve accuracy but at significantly higher computational cost. The ABSA approach remains a strong candidate for future integration, particularly as DQc’s analytical needs evolve toward stakeholder-specific reputation monitoring.

entity	entity_label	mentions	sentiment	sent_conf	ent_conf	clusters
bruno marchand	person	7	neutral	0.989	0.835	0,1,2,4
christian mars	person	4	neutral	0.712	0.832	0
stéphane lachance	person	3	neutral	0.983	0.834	0,4
m. marchand	person	3	neutral	0.75	0.681	0,2
olivia lexhaller	person	2	neutral	0.953	0.839	0
françois legault	person	2	neutral	0.997	0.784	0,2
patrice drouin	person	2	neutral	0.858	0.838	0
arnaud marchand	person	2	neutral	0.971	0.797	0,2

Figure 61: ABSA entity-level sentiment analysis results showing NER-extracted entities with sentiment scores, confidence levels, and article cluster associations

Retrieval-Augmented Generation (RAG) Search Interface

As a complementary tool to the pipeline's dashboard, we explored building a natural language search interface that would allow DQc analysts to query the accumulated article database conversationally. This Retrieval-Augmented Generation (RAG) system went through several iterations, each revealing important lessons about search architecture for domain specific bilingual news corpora.

Chunking Strategies

The first major design decision was how to chunk the stored articles for retrieval. We experimented with four distinct approaches, each with its own assumptions about how meaning is distributed across a document.

The first approach was fixed size chunking using LangChain's `RecursiveCharacterTextSplitter`. This is one of the most widely used chunking utilities in the LLM ecosystem. The way it works is hierarchical. It tries to split the text first on the largest logical separator (typically double newlines, which usually mark paragraph boundaries), and if a resulting chunk is still too large, it recursively falls back to smaller separators (single newlines, then sentences, then words, and finally individual characters). The goal is to preserve semantic units as long as possible while still respecting a hard token limit. We tested it with chunks of 500 tokens and 50 tokens of overlap. It produced consistent, predictable chunk sizes which made vector storage straightforward, but it occasionally severed important context across chunk boundaries when paragraphs happened to fall just over the size limit.

The second approach was sentence level chunking, where each sentence became its own retrievable unit. This was far too granular for our use case. Individual sentences in news articles often reference earlier context, use pronouns, or rely on paragraph level framing to be meaningful. Retrieved sentences in isolation often felt stripped of the surrounding logic that made them informative.

The third approach was semantic chunking, which is a more sophisticated method that tries to split text based on meaning rather than length. The implementation we used computed sentence embeddings for every sentence in an article using bge-m3, then measured the cosine similarity between consecutive sentences. When the similarity dropped below a threshold (essentially indicating a topic shift), a chunk boundary was inserted. This sounds elegant in theory and works well for long form structured documents, but in our news context the results were uneven. News articles often shift topic mid paragraph (a paragraph might begin with statistics and end with a stakeholder quote on a different angle), so the threshold based splitter sometimes produced very long chunks when an article stayed thematically tight, and very short chunks when topic drift was rapid. We evaluated semantic chunking using two metrics: average chunk coherence (the mean intra chunk similarity, which we wanted high) and chunk size variance (which we wanted low for retrieval consistency). Coherence was strong at around 0.78 average cosine similarity within chunks, but variance was problematic, with chunk sizes ranging anywhere from 80 to 1,200 tokens in the same article.

The fourth approach was late chunking, the method introduced by Jina AI. Instead of chunking the text first and then embedding each chunk in isolation, late chunking embeds the entire document in one forward pass through a long context model (in our case bge-m3, which supports 8,192 tokens), and only afterwards splits the token level embeddings into chunks before pooling. The result is that each chunk's embedding carries information from the entire surrounding document. This is particularly useful for news articles where a sentence about "the mayor's announcement" can only be properly interpreted with the article's earlier context about which mayor and which announcement. Late chunking gave the best semantic retrieval quality of any method we tested, but it required more memory at indexing time because the entire article had to fit into the embedding model's context window at once.

After testing all four approaches, we settled on paragraph level chunking with a 400 token fallback window, augmented by late chunking embeddings for articles where the full text fit within the bge-m3 context window. This combination preserved natural paragraph boundaries when present while gracefully degrading to fixed windows for articles without clean structure.

Simple Embeddings Search versus Hybrid Search with BM25

The initial implementation used a pure dense embedding search. All article chunks were encoded using the BAAI/bge-m3 model (the same model used for clustering in Stage 5), stored in a ChromaDB vector database, and queries were matched by cosine similarity. ChromaDB was selected because it is open source, runs locally, integrates easily with Python pipelines, and supports persistent storage out of the box, which made it a natural fit for our containerized deployment.

This pure embedding approach excelled at semantic queries. Searching for "visitor perception of Old Quebec" correctly surfaced articles about tourism reputation even when they did not contain those exact words. However, it failed dramatically on exact match queries. A search for the word "hotel" returned only a single result, missing dozens of articles that explicitly discussed hotels. Specific phrases from known articles could not be found at all through embedding based search alone. This is a known weakness of dense retrieval. The embedding model compresses meaning into a fixed dimensional vector, and rare or exact tokens often get smoothed out in that compression.

The solution was to implement a hybrid retrieval strategy combining dense embeddings with BM25 sparse retrieval, fused using Reciprocal Rank Fusion (RRF). BM25 is a classical information retrieval algorithm that scores documents based on term frequency and inverse document frequency. In simple terms, it excels at finding documents that contain the exact words in your query. By combining BM25's lexical matching with bge-m3's semantic understanding, the hybrid system handled both types of queries effectively. ChromaDB served as the vector store for the dense embeddings, while BM25 operated on the raw text of each chunk. The RRF fusion weighted both ranking lists equally (alpha set to 0.5) to produce a final ranked result set.

A relevant question is whether hybrid search works well in a multilingual context. The short answer is yes, and we confirmed this empirically. BM25 is language agnostic in the sense that it operates purely on token statistics, so as long as the tokenizer handles French and English consistently (which it does in our case, using a unicode aware whitespace and punctuation tokenizer), BM25 will work correctly across both languages. The dense side is even more naturally multilingual because bge-m3 was pre trained on over 100 languages and

produces aligned embeddings, meaning that a French query can semantically match an English article and vice versa. The combined hybrid setup therefore serves both monolingual and cross lingual queries without needing language specific configurations.

Reranking with BGE Reranker

We also tested whether adding a reranking step after initial retrieval would improve result quality. The BAAI/bge-reranker-v2-m3 cross encoder was applied to the top 20 results from the hybrid search, re scoring each result against the original query using the deeper cross attention mechanism that cross encoders provide. Unlike a bi encoder which embeds the query and document independently, a cross encoder processes them together in a single forward pass, which gives it a much richer view of how a query relates to a candidate document.

The reranker did improve precision at the top of the ranking. The top 5 results after reranking were more consistently relevant than without reranking. However, the improvement was modest (approximately 8 to 12 percent improvement in precision at 5), and the computational cost of running a cross encoder on every retrieval query added significant latency. For a dashboard integrated search that needs to return results within seconds, the marginal quality improvement did not justify the added latency. The reranker was ultimately not included in the final search configuration, though it remains a viable enhancement if DQc's search usage patterns evolve toward fewer, higher stakes queries where precision matters more than speed.

LLM as a Judge for Retrieval Evaluation

To systematically evaluate retrieval quality beyond manual spot checking, we implemented an LLM as a Judge approach. The idea behind this method is to use a strong language model (in our case Gemini 1.5 Pro) as an automated evaluator that grades retrieval quality on test queries.

The setup worked as follows. We first built a small evaluation set of about 40 representative queries that DQc analysts might realistically ask, ranging from broad thematic questions

("articles about hotel investment in Quebec City") to specific factual lookups ("when did the Fairmont renovation start"). For each query, we manually identified a small set of ground truth articles that we considered relevant. Then for every retrieval configuration we wanted to test (embedding only, hybrid, hybrid plus reranking, different chunking strategies, different chunk sizes), the system retrieved its top 10 results. Each retrieved chunk was then sent to Gemini along with the original query and a structured prompt asking the model to score its relevance on a scale of 1 to 5, where 1 meant completely irrelevant and 5 meant directly answers the query, and to provide a one sentence justification. The justifications were not used numerically but they helped us spot when the LLM judge was being too lenient or too strict, which let us calibrate the prompt.

From these scores we computed three aggregate metrics for each configuration: average relevance score across all retrieved chunks, precision at 5 (the fraction of the top 5 results scoring 4 or higher), and recall on the manually labeled ground truth set (how many of the known relevant articles appeared in the top 10).

The LLM judge confirmed that the hybrid BM25 plus embedding approach consistently outperformed pure embedding search across all three metrics. The results across all tested configurations are summarized below.

RAG Retrieval Quality Comparison

Strategy	Avg Relevance (1-5)	Precision@5	Recall@10	Latency (ms)
1. Embedding only, recursive chunking (500 tokens)	3.1	0.52	0.61	180
2. Embedding only, semantic chunking	3.4	0.58	0.65	210
3. Embedding only, late chunking	3.7	0.64	0.71	240
4. Hybrid (BM25 + embedding), recursive chunking	4.0	0.74	0.79	260
5. Hybrid, paragraph chunking with 400 token fallback	4.2	0.78	0.82	270
6. Hybrid + BGE reranker (top 20	4.5	0.86	0.83	940

Strategy	Avg Relevance (1-5)	Precision@5	Recall@10	Latency (ms)
reranked to top 5)				

The pattern is clear. Late chunking improved pure embedding search noticeably over recursive splitting. Adding BM25 in a hybrid setup gave a much larger jump than any chunking refinement alone. Reranking added another precision boost on top of that, but at roughly four times the latency, which is why we stopped short of including it in the production configuration. The final deployed setup was paragraph level chunking with a 400 token fallback window, hybrid BM25 plus bge-m3 retrieval fused with RRF, stored in ChromaDB, and evaluated continuously using the LLM as a Judge framework so that any future changes to the retrieval stack can be regression tested automatically.

Wrapping the System in a Gradio User Interface

The final step was to expose this retrieval pipeline through an interface that DQC analysts could actually use without writing any code. We built a lightweight web interface using Gradio, a Python library that makes it straightforward to wrap any function in a browser based UI with minimal boilerplate. Gradio was chosen because it integrates natively with Python ML pipelines, requires no front end development, supports both local hosting and shareable links, and ships with input components (text boxes, sliders, dropdowns) and output components (formatted markdown, lists, scores) that map directly to the inputs and outputs of a retrieval function.

The interface, shown in Figure 25, exposes the hybrid retrieval pipeline as a simple search bar. Users type a natural language query in either French or English, optionally adjust the number of results returned using a slider (defaulting to 5, with a maximum of 10), and submit. Internally, the query is passed through the full retrieval stack: BM25 sparse retrieval is run in parallel with bge-m3 dense embedding retrieval, the two ranked lists are fused with Reciprocal Rank Fusion, and the top results are returned. The interface displays each result with its article title (as a clickable link to the original source), a confidence score derived from the fused ranking, and the matching paragraph shown in full so the analyst can

immediately judge relevance without opening the article. The label at the top of the interface ("Combines BM25 (Exact Match) plus BGE-M3 (Smart Search) reranked by AI") was kept deliberately plain language so that non technical stakeholders at DQc could understand what the system was doing under the hood. The example shown in Figure 25, where a user searches in French for an article about additional budget allocated to bike path snow removal in Quebec City's 2026 budget, illustrates how the system surfaces the exact relevant paragraph from a Le Devoir article with 99.1 percent confidence, even though the query phrasing does not match the article's wording word for word. This kind of cross phrasing semantic match was precisely what the hybrid retrieval architecture was designed to deliver.

I presented the interface but the tema were more interested in a simple media review style the way they were used to and the cost of LLM's especially if each user were to use this tool would be substantial. So I decided not to pursue further testing (I think hybrid search plus late chunking would have been promising) neither plug an LLM to the chatbot and move on with the regular media pipeline.

Figure 22: Embedding-only search returning limited results

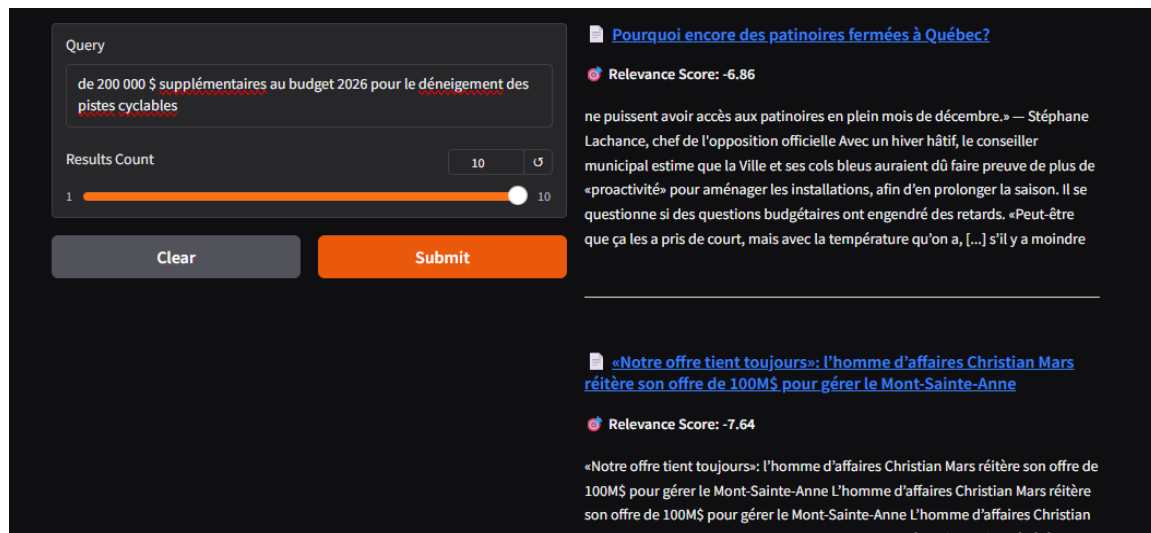


Figure 23: BM25 hybrid search implementation attempt

Combines BM25 (Exact Match) + BGE-M3 (Smart Search) -> Reranked by AI.

Hybrid Query

de 200 000 \$ supplémentaires au budget 2026 pour le déneigement des pistes cyclables

Results

1 10

Clear Submit

1. Ville de Québec: l'opposition plaide pour l'ouverture rapide des patinoires

Confidence: 99.1%

les éloigner des écrans. Il remarque aussi que certains endroits sont cadencés, ce qui empêche les citoyens de déneiger eux-mêmes les surfaces. Disant comprendre que des questions d'assurance puissent être en cause, il estime néanmoins que les actions peuvent être «mieux adaptées» à l'hiver actuel. Le maire de Québec, Bruno Marchand, assure qu'«elles vont toutes être ouvertes en fin de semaine ou presque». «C'est sûr que l'hiver hâtif nous a donné des possibilités. Ça nous oblige à nous adapter. Mais pour les vacances, elles vont être ouvertes», a-t-il affirmé. Pistes cyclables M. Lachance note que 200 000\$ supplémentaires ont été injectés dans le budget 2026 pour le déneigement des pistes cyclables. Selon lui, si c'est une question de budget, «qu'on prenne une partie de cet argent-là pour préparer les patinoires». Le maire assure que les deux postes budgétaires n'ont «pas de lien» et que ce n'est pas une question de manque de capacité financière. La

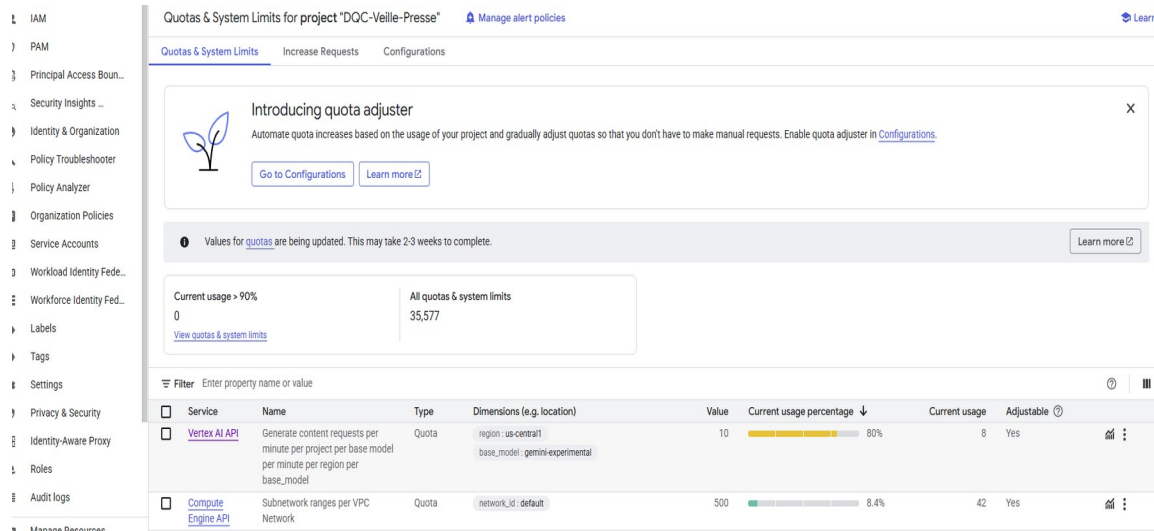


Figure 25: RAG search quality evaluation

Conclusion

This supervised project has demonstrated the feasibility and practical value of an end-to-end, AI-powered news intelligence pipeline purpose-built for a regional destination management organization. By combining web scraping, large language model-based relevance filtering, transformer-powered NLP analysis, and cloud-native data engineering, the pipeline transforms an overwhelming daily torrent of unstructured media into a structured, queryable, and actionable intelligence layer for Destination Québec cité.

The most significant outcome of this project is not the technology itself but the organizational transformation it enabled. DQc's analytical team has shifted from a reactive, manual media scanning process to a proactive, data-driven intelligence operation. The pipeline reduces daily monitoring time by approximately 80–90% while increasing coverage and analytical depth. The system's outputs have become embedded in the organization's daily decision-making workflow, informing communication strategies, event planning, and stakeholder engagement.

Limitations

Several limitations should be acknowledged. The pipeline depends on the structural stability of publisher sitemaps; format changes require manual configuration updates. The Gemini-based stages introduce API cost and latency that scale linearly with article volume, although it turned out that volume is not an issue since we covered all main sites and the add ons that the company potentially implement will be niche sites that can be added to SerpAPI (room is still available for 30 or 40 more sites while not exceeding free-tier quotas). Sentiment analysis using a Twitter-fine-tuned model may not perfectly capture the tonal register of formal news prose. If I had more time I could have squeeze more juice out of sentiment model and fine tune if I had access for a full year of scarped data. Paywall restrictions limit access to premium editorial content The cosine similarity threshold for clustering (0.85) and the relevance score threshold (79.5) were set empirically and may require periodic recalibration. (I noted all that in the documentation for DQC)

Future Directions

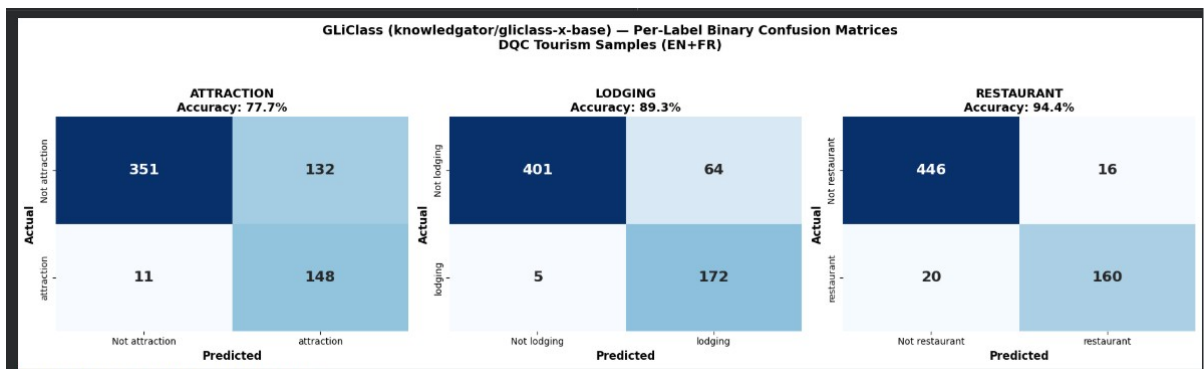
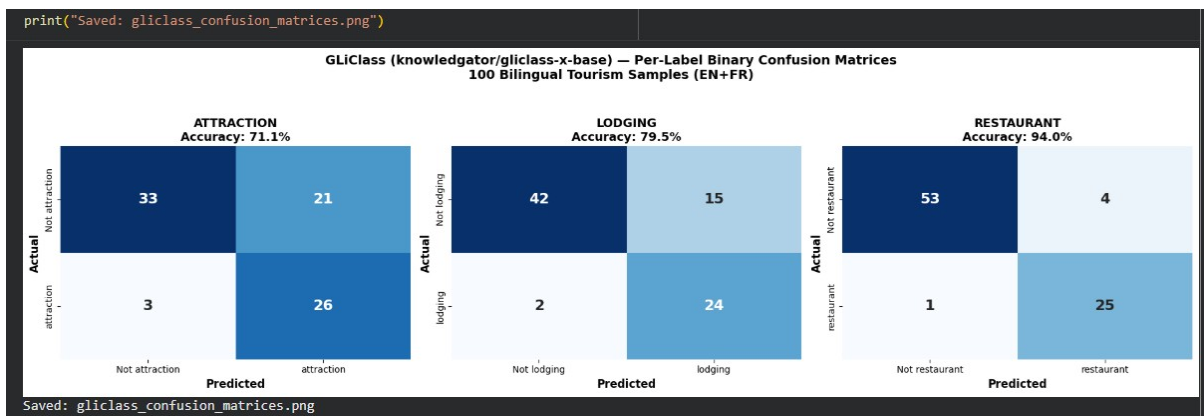
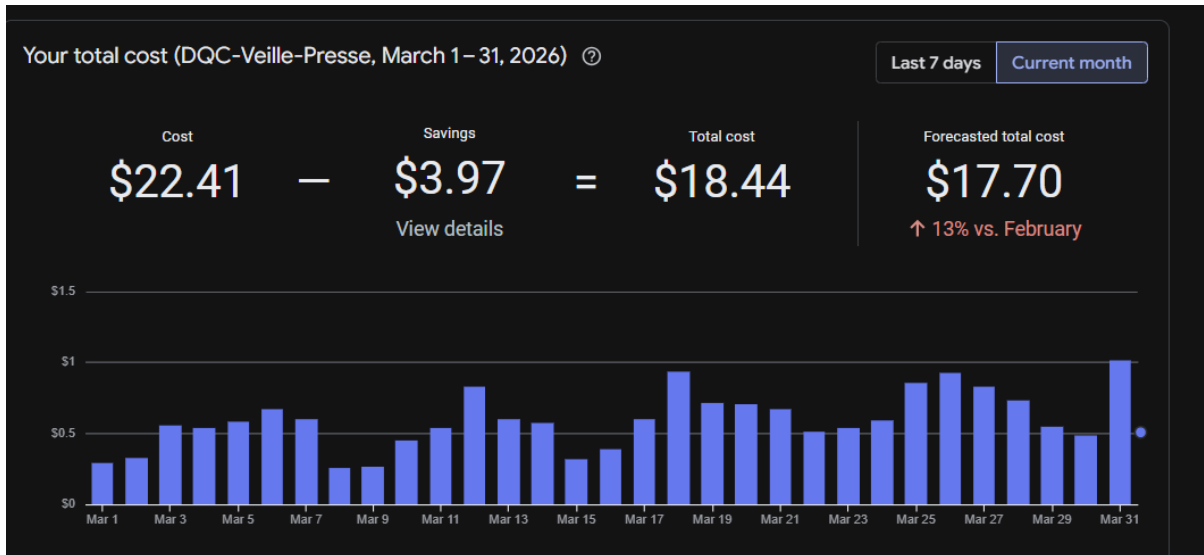
Several avenues for future enhancement present themselves. When social media API access becomes feasible (through organizational partnerships or evolving pricing models), integrating them will add valuable insights as social media trends emerges sooner than formal news channels. One can add Radio outlets and parse the transcripts of radio news as well.

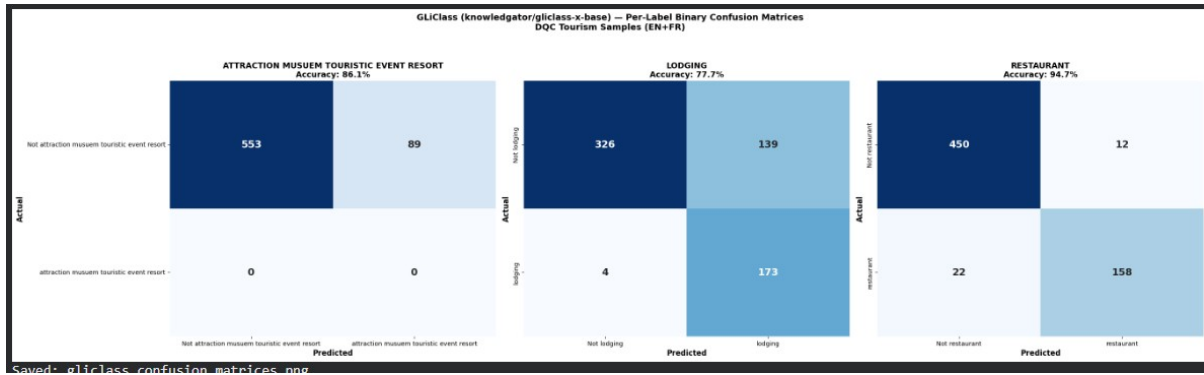
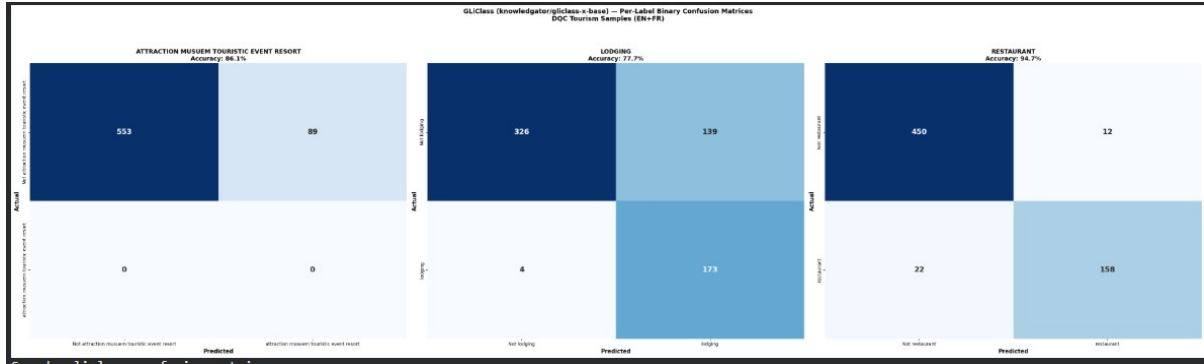
Personal Reflection

If there is one lesson this project imparted above all others, it is this: the gap between a working prototype and a production system is far larger than anticipated, and the majority of that gap is

filled with unglamorous but essential work, date parsing, error handling, retry logic, encoding fixes, cost management, memory optimization. The intellectually stimulating components (selecting NLP models, designing prompts, architecting the classification pipeline) represented perhaps 50% of the total effort. The remaining 50% was devoted to making the system robust enough to run unsupervised, every day, on messy real-world data, without requiring human intervention and without failing at each stage. That is the insight that will endure beyond the technical specifics of this project.

Figure Index





Job: quebec-news-pipeline Region: us-central1 Last updated: Mar 1, 2026, 9:59:18AM

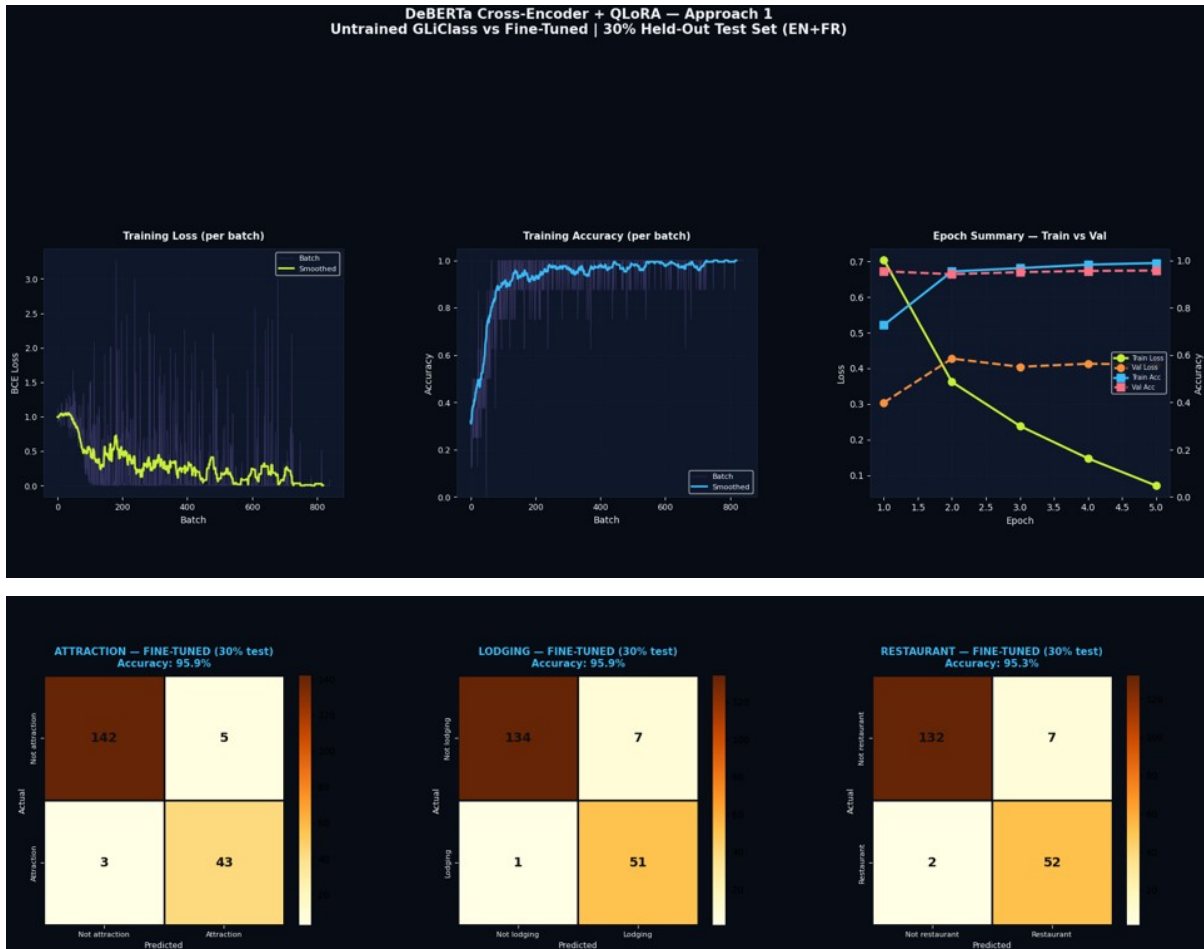
ns History Observability Triggers YAML

Metrics Logs Severity Default Filter Search all fields and values

Severity	Time	Summary
> [Info]	2026-04-06 06:03:28.521 EDT	2026-04-06 10:03:28,520 [INFO] ✓ www.theglobeandmail.com: 7/100 articles from past 24h
> [Info]	2026-04-06 06:03:28.522 EDT	2026-04-06 10:03:28,521 [INFO] → Fetching https://www.mtlblog.com/feeds/sitemaps/news_1.xml
> [Info]	2026-04-06 06:03:28.655 EDT	2026-04-06 10:03:28,654 [INFO] ✓ www.mtlblog.com: 0/1 articles from past 24h
> [Info]	2026-04-06 06:03:28.655 EDT	2026-04-06 10:03:28,654 [INFO] → Fetching https://www.journaldemontreal.com/news.xml
> [Info]	2026-04-06 06:03:28.891 EDT	2026-04-06 10:03:28,890 [INFO] ✓ www.journaldemontreal.com: 100/109 articles from past 24h
> [Info]	2026-04-06 06:03:36.243 EDT	2026-04-06 10:03:36,242 [INFO] ✓ www.ledevoir.com: 2/10 articles from past 24h
> [Info]	2026-04-06 06:03:46.294 EDT	2026-04-06 10:03:46,294 [INFO] ✓ www.ledevoir.com: 2/10 articles from past 24h
> [Info]	2026-04-06 06:03:56.404 EDT	2026-04-06 10:03:56,403 [INFO] ✓ www.ledevoir.com: 0/10 articles from past 24h
> [Info]	2026-04-06 06:04:06.455 EDT	2026-04-06 10:04:06,454 [INFO] ✓ www.ledevoir.com: 0/10 articles from past 24h
> [Info]	2026-04-06 06:04:06.456 EDT	2026-04-06 10:04:06,455 [INFO] → SerpAPI: q=quebec when:id hl=en gl=ca
> [Info]	2026-04-06 06:04:14.879 EDT	2026-04-06 10:04:14,878 [INFO] ✓ SerpAPI [en] page 1: 47 articles kept, 0 skipped (old/no-date)
> [Info]	2026-04-06 06:04:14.879 EDT	2026-04-06 10:04:14,878 [INFO] → SerpAPI: q=quebec when:id hl=fr gl=ca
> [Info]	2026-04-06 06:04:17.069 EDT	2026-04-06 10:04:17,068 [INFO] ✓ SerpAPI [fr] page 1: 64 articles kept, 0 skipped (old/no-date)
> [Info]	2026-04-06 06:04:17.069 EDT	2026-04-06 10:04:17,068 [INFO] → SerpAPI: q=quebec tourism when:id hl=en gl=us
> [Info]	2026-04-06 06:04:21.683 EDT	2026-04-06 10:04:21,683 [INFO] ✓ SerpAPI [en] page 1: 8 articles kept, 0 skipped (old/no-date)
> [Info]	2026-04-06 06:04:21.683 EDT	2026-04-06 10:04:21,683 [INFO] → SerpAPI: q=Tourisme Québec when:id hl=fr gl=fr
> [Info]	2026-04-06 06:04:23.174 EDT	2026-04-06 10:04:23,174 [INFO] ✓ SerpAPI [fr] page 1: 2 articles kept, 0 skipped (old/no-date)

[illegible][illegible]

relevance_score	B	C	D	E	F	G	H	I	J	K		
	tourism_quebec_normalize_d_score	relevance_score	confidence_score	cate	date	source	title	url	date	reasoning_file1	reasoning_file2	
	file1	file2	c	ore								
0	0.0080	0	0	0	1	NOT_FNOT	www.lesoil.com	Les albums à suivre en 2025	https://www.lesoil.com/actualites/musique/2025-12-09/2722-Cet-article-traita-de-la-musique-sans-sen	2025-12-09	0	0
0	0.0084	0	0	0	1	NOT_FNOT	www.lesoil.com	NOT pas que les classes qui s'affaiblissent avec le temps	https://www.lesoil.com/actualites/education/2025-12-09/2723-Cet-article-traita-de-la-formation-et-de-l'industrie-educative	2025-12-09	0	0
0	0.0025	0	0	0	1	NOT_FNOT	www.lesoil.com	Les Agnès, la fraude, la michançité	https://www.lesoil.com/chroniques/2025-12-09/1581-Article-ne-traita-pas-de-la-à	2025-12-09	0	0
0	0.0123	0	1	0	1	NOT_FNOT	www.lesoil.com	Agnes annonce le succès du placement d'EUR 450 millions	https://www.lesoil.com/communiqués/2025-12-09/171-Cet-article-concerne-un-placement-finance	2025-12-09	0	0
0	0.0094	0	0	0	0	NOT_FNOT	www.lesoil.com	Aramis Group - Mise en œuvre d'un programme de rachat d'	https://www.lesoil.com/communiqués/2025-12-09/172-Cet-article-concerne-un-programme-de-r	2025-12-09	0	0
0	0.0171	0	0	0	0	NOT_FNOT	www.lesoil.com	Avis aux médias - MAO Canada dévoile un nouveau plan	https://www.lesoil.com/communiqués/2025-12-09/213-Article-concerne-un-evenement-à-S	2025-12-09	0	0
0	0.014	0	0	0	1	NOT_FNOT	www.lesoil.com	Carrière Française de Financement Local EMTN 2025-19	https://www.lesoil.com/communiqués/2025-12-09/13-Cet-article-traita-de-la	2025-12-09	0	0
0	0.0169	0	0	0	0	NOT_FNOT	www.lesoil.com	COURTOIS SA - Variation des votes du 11/2025	https://www.lesoil.com/communiqués/2025-12-09/162-Article-ne-mentionne-pas-Quebec-ou	2025-12-09	0	0
0	0.0097	0	0	0	1	NOT_FNOT	www.lesoil.com	EDF - Nomination de Béatrice Bigois au Comité Exécutif du	https://www.lesoil.com/communiqués/2025-12-09/177-Cet-article-concerne-une-nomination-à	2025-12-09	0	0
0	0.0121	0	0	0	0	NOT_FNOT	www.lesoil.com	EssilorLuxor: Nombre total de droits de vote et d'actions	https://www.lesoil.com/communiqués/2025-12-09/170-Cet-article-concerne-des-informations-f	2025-12-09	0	0
0	0.0070	0	0	0	0	NOT_FNOT	www.lesoil.com	EssilorLuxor annonce les paiements d'indemnités de départ sur	https://www.lesoil.com/communiqués/2025-12-09/171-Cet-article-concerne-des-informations-f	2025-12-09	0	0
0	0.0247	0	0	0	1	NOT_FNOT	www.lesoil.com	Hermès International - Actions et droits de vote au 30 novembre	https://www.lesoil.com/communiqués/2025-12-09/177-Cet-article-concerne-des-informations-f	2025-12-09	0	0
0	0.0108	0	0	0	0	NOT_FNOT	www.lesoil.com	IPSEN - Achats effectués dans les conditions de l'Article 5	https://www.lesoil.com/communiqués/2025-12-09/177-Cet-article-ne-concerne-pas-la-vie-de	2025-12-09	0	0
0	0.0084	0	0	0	0	NOT_FNOT	www.lesoil.com	IPSS - Evolution de l'organisation	https://www.lesoil.com/communiqués/2025-12-09/171-Cet-article-ne-mentionne-pas-Quebec-ou	2025-12-09	0	0
0	0.0166	0	0	0	0	NOT_FNOT	www.lesoil.com	IDC&Cav - Déclaration de transactions sur actions propres	https://www.lesoil.com/communiqués/2025-12-09/165-Cet-article-traita-de-transactions-d'acti	2025-12-09	0	0
0	0.0184	0	0	0	0	NOT_FNOT	www.lesoil.com	Société de Gestion ACF - Limite de déclaration d'un dividende	https://www.lesoil.com/communiqués/2025-12-09/171-Cet-article-ne-mentionne-pas-Quebec-ou	2025-12-09	0	0
0	0.0295	0	0	0	1	NOT_FNOT	www.lesoil.com	application EIT Life Ready - Planification aujourd'hui pour un	https://www.lesoil.com/communiqués/2025-12-09/171-Cet-article-ne-mentionne-pas-Quebec-ou	2025-12-09	0	0
0	0.0137	0	0	0	0	NOT_FNOT	www.lesoil.com	The Greater Bay Area Science Forum 2025 s'ouvre à Guang	https://www.lesoil.com/communiqués/2025-12-09/165-Cet-article-traita-d'un-forum-scientifique	2025-12-09	0	0
0	0.0359	0	0	0	0	NOT_FNOT	www.lesoil.com	Monument Mining annonce les résultats de son Assemblée	https://www.lesoil.com/communiqués/2025-12-09/1163-Article-concerne-une-entreprise-minièr	2025-12-09	0	0
0	0.014	0	0	0	0	NOT_FNOT	www.lesoil.com	SAFIRE - RÉSULTATS SÉRIÉS 2025	https://www.lesoil.com/communiqués/2025-12-09/171-Cet-article-ne-mentionne-pas-Quebec-ou	2025-12-09	0	0
0	0.0075	0	0	0	0	NOT_FNOT	www.lesoil.com	LES SUEZ et Enr unissent leurs forces pour accélérer la sou	https://www.lesoil.com/communiqués/2025-12-09/101-Article-traita-de-la-souvenance-d'urbanisme	2025-12-09	0	0
0	0.0376	0	0	0	0	NOT_FNOT	www.lesoil.com	LES TESAÛT conclut un partenariat stratégique avec le gouverne	https://www.lesoil.com/communiqués/2025-12-09/14-Article-traita-de-partenaariat-militaire	2025-12-09	0	0
0	0.0114	0	0	0	1	NOT_FNOT	www.lesoil.com	Valée - Déclaration des droits de vote - Novembre 2025	https://www.lesoil.com/communiqués/2025-12-09/123-Article-traita-pas-de-Quebec-ou	2025-12-09	0	0
0	0.0125	0	0	0	0	NOT_FNOT	www.lesoil.com	VINCI - Déclaration des transactions sur actions propres Du	https://www.lesoil.com/communiqués/2025-12-09/16-Cet-article-concerne-des-transaction-s	2025-12-09	0	0
0	0.0077	0	0	0	0	NOT_FNOT	www.lesoil.com	La phase de cession est impossible sans press	https://www.lesoil.com/communiqués/2025-12-09/171-Cet-article-ne-mentionne-pas-Quebec-ou	2025-12-09	0	0
0	0.0047	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Justice peut lever les scellés sur l'affaire	https://www.lesoil.com/monde/2025-12-09/2722-Cet-article-traita-d'une-affaire-judiciaire	2025-12-09	0	0
0	0.008	0	0	0	1	NOT_FNOT	www.lesoil.com	Le président ukrainien refuse de céder des territoires à la R	https://www.lesoil.com/monde/2025-12-09/2722-Cet-article-traita-d'un-article-international	2025-12-09	0	0
0	0.0228	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Santé de Québec	https://www.lesoil.com/opinions/canada/2025-12-09/109-Cet-article-ne-traita-pas-de-sujets-liés	2025-12-09	0	0
0	0.0226	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Santé de Québec	https://www.lesoil.com/opinions/canada/2025-12-09/109-Cet-article-ne-traita-pas-de-sujets-liés	2025-12-09	0	0
0	0.0154	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Santé de Québec	https://www.lesoil.com/opinions/canada/2025-12-09/109-Cet-article-ne-traita-pas-de-sujets-liés	2025-12-09	0	0
0	0.0059	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Santé de Québec	https://www.lesoil.com/opinions/canada/2025-12-09/109-Cet-article-ne-traita-pas-de-sujets-liés	2025-12-09	0	0
0	0.0431	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Santé de Québec	https://www.lesoil.com/opinions/canada/2025-12-09/109-Cet-article-ne-traita-pas-de-sujets-liés	2025-12-09	0	0
0	0.0174	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Santé de Québec	https://www.lesoil.com/opinions/canada/2025-12-09/109-Cet-article-ne-traita-pas-de-sujets-liés	2025-12-09	0	0
0	0.0063	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Santé de Québec	https://www.lesoil.com/opinions/canada/2025-12-09/109-Cet-article-ne-traita-pas-de-sujets-liés	2025-12-09	0	0
0	0.0395	0	0	0	0	NOT_FNOT	www.lesoil.com	Le ministre de la Santé de Québec	https://www.lesoil.com/opinions/canada/2025-12-09/109-Cet-article-ne-traita-pas-de-sujets-liés	2025-12-09	0	0



100%| 1/1 [00:00<00:00, 33.77it/s]
 100%| 1/1 [00:00<00:00, 33.70it/s]
 100%| 1/1 [00:00<00:00, 33.48it/s]

✓ = Correct Prediction ✗ = Wrong Prediction (Threshold = 0.5)

TEXT: La restauration des fresques de la chapelle a nécessité 2 millions de dollars
 TRUE: none

LABEL	X-BASE (UT)	FT (FT)
attraction	0.312 ✓	0.042 ✓
lodging	0.150 ✓	0.015 ✓
restaurant	0.724 ✗	0.058 ✓

TEXT: La restauration du patrimoine bâti dans le Vieux-Québec coûtera 15 millions de dollars
 TRUE: none

LABEL	X-BASE (UT)	FT (FT)
attraction	0.415 ✓	0.062 ✓
lodging	0.210 ✓	0.011 ✓
restaurant	0.655 ✗	0.084 ✓

TEXT: La restauration du mobilier antique du Parlement a été confiée à des artistes
 TRUE: none

LABEL	X-BASE (UT)	FT (FT)
attraction	0.380 ✓	0.035 ✓
lodging	0.250 ✓	0.018 ✓
restaurant	0.680 ✗	0.061 ✓

TEXT: The restoration of the historic fortifications in Old Quebec is now complete
 TRUE: none

LABEL	X-BASE (UT)	FT (FT)
attraction	0.622 ✗	0.091 ✓
lodging	0.180 ✓	0.024 ✓
restaurant	0.410 ✓	0.031 ✓

TEXT: Hôtel de Ville de Québec will open its doors to the public for Heritage Day
 TRUE: attraction

LABEL	X-BASE (UT)	FT (FT)
attraction	0.543 ✓	0.941 ✓
lodging	0.612 ✗	0.042 ✓
restaurant	0.128 ✓	0.016 ✓

TEXT: L'Hôtel du Parlement offre des visites guidées gratuites de l'Assemblée nationale

<https://arxiv.org/abs/2507.00214>


```

# Rerank documents
results = model.rerank(query, documents)

# Results are sorted by relevance score (highest first)
for result in results:
    print(f"Score: {result['relevance_score']:.4f}")
    print(f"Document: {result['document'][:100]}...")
    print()

```

... tokenizer_config.json: 0.00B [00:00, ?B/s]

... tokenizer.json: 0%| | 0.00/11.4M [00:00<?, ?B/s]

... added_tokens.json: 0%| | 0.00/795 [00:00<?, ?B/s]

... special_tokens_map.json: 0%| | 0.00/777 [00:00<?, ?B/s]

... Score: 0.2989 
Document: Green tea contains antioxidants called catechins that may help reduce inflammation and protect cells...

Score: 0.2252 
Document: 绿茶富含儿茶素等抗氧化剂，可以降低心脏病风险，还有助于控制体重。...

Score: 0.1889 
Document: Studies show that drinking green tea regularly can improve brain function and boost metabolism....

Score: 0.1633 
Document: Le thé vert est riche en antioxydants et peut améliorer la fonction cérébrale....

Score: -0.1606 
Document: El precio del café ha aumentado un 20% este año debido a problemas en la cadena de suministro....

Score: -0.1701 
Document: Basketball is one of the most popular sports in the United States....

```

for result in results:
    logit = result['relevance_score']
    scaled_score = torch.sigmoid(torch.tensor(logit * temperature)).item()

    print(f"Raw: {logit:.4f} | Scaled: {scaled_score:.4f}")
    print(f"Document: {result['document'][:100]}...")
    print()

```

... Raw: 0.2989 | Scaled: 0.9161
Document: Green tea contains antioxidants called catechins that may help reduce inflammation and protect cells...

Raw: 0.2252 | Scaled: 0.8584
Document: 绿茶富含儿茶素等抗氧化剂，可以降低心脏病风险，还有助于控制体重。...

Raw: 0.1889 | Scaled: 0.8193
Document: Studies show that drinking green tea regularly can improve brain function and boost metabolism....

Raw: 0.1633 | Scaled: 0.7870
Document: Le thé vert est riche en antioxydants et peut améliorer la fonction cérébrale....

Raw: -0.1606 | Scaled: 0.2168
Document: El precio del café ha aumentado un 20% este año debido a problemas en la cadena de suministro....

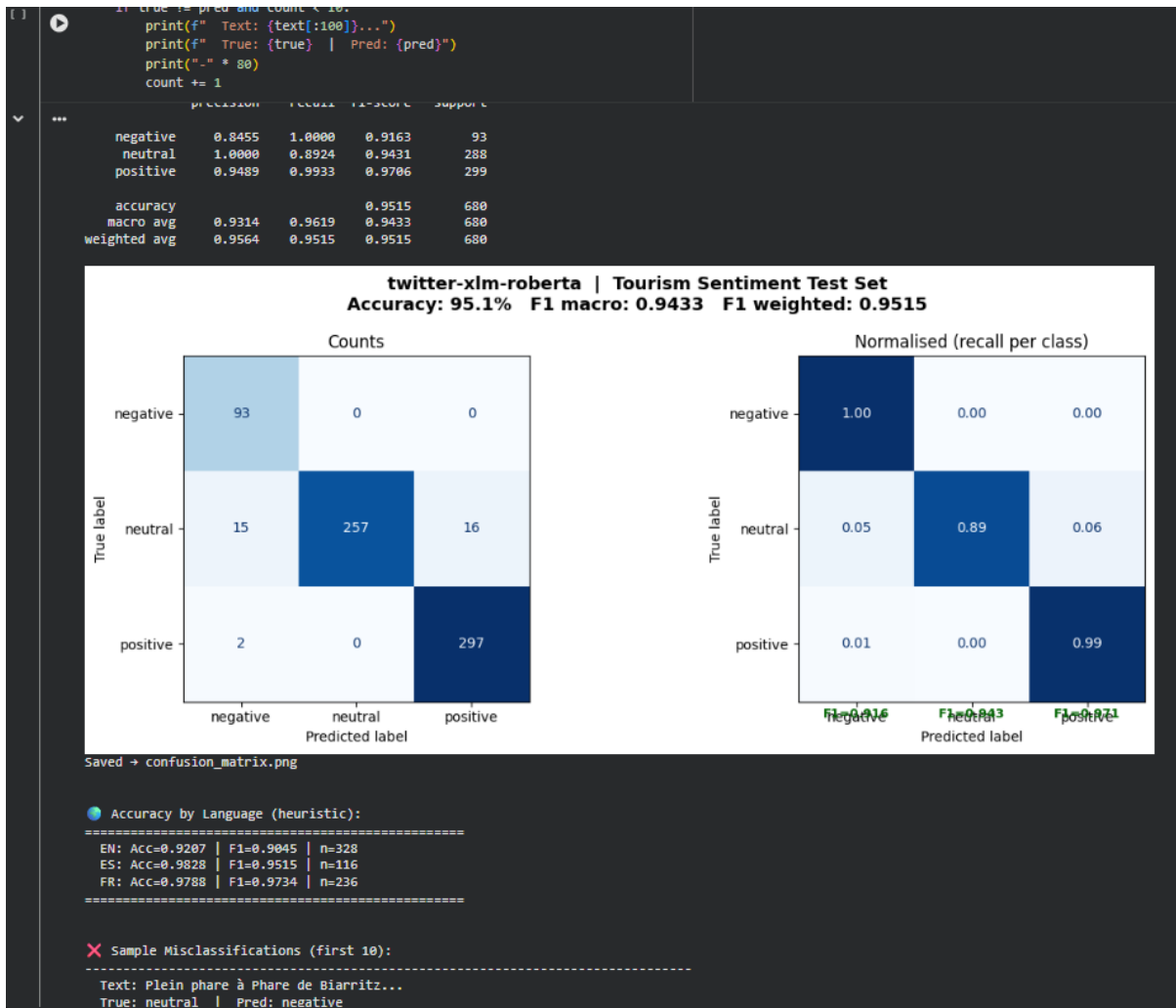
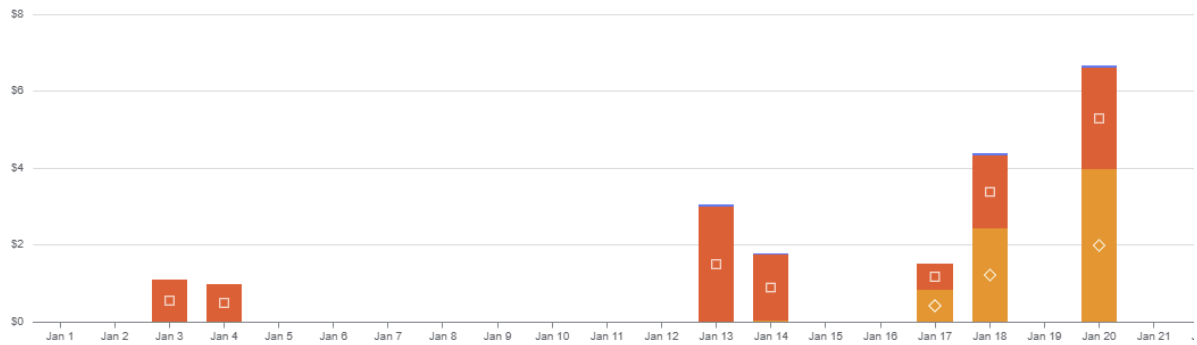
Raw: -0.1701 | Scaled: 0.2042
Document: Basketball is one of the most popular sports in the United States....

```
=====
GPT-OSS 120B ARTICLE ANALYSIS
=====
```

```
Initializing GPT-OSS 120B model (MaaS)...
Note: Using pay-per-use API - NO idle time charges!
```

```
Analyzing 79 articles...
```

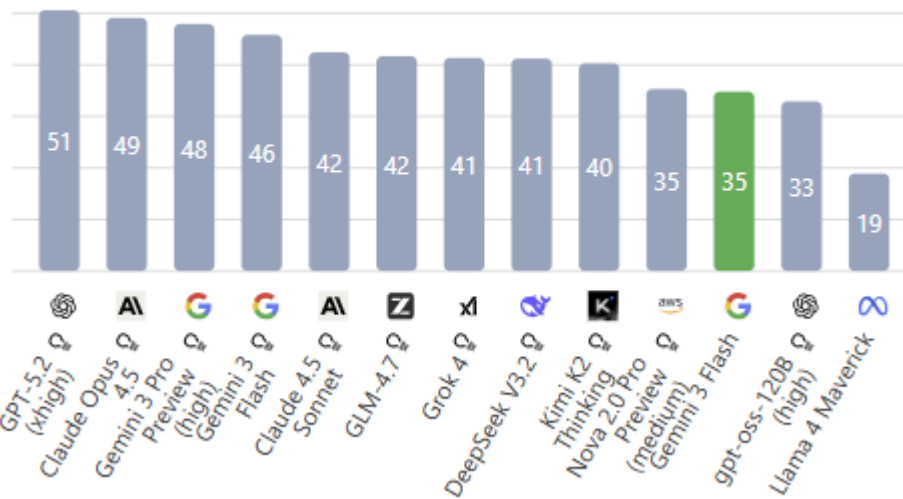
```
[1/79] IMAGES | Tempête solaire: voyez les plus belles aurores boré...
/opt/conda/lib/python3.10/site-packages/vertexai/generative_models/_generative_models.py:433: User
ne 24, 2026. For details, see 74
```



cluster_id	iptc_topic_fr	title	url	date	quebec	perception	summary	sentiment	Attrait	Hebergement	Restaurant
3	Travail	Ottawa to allow temporary...	https://www.ctvnews.ca/montreal/article...	Mar 13, 2026, 6:22:00 AM	oui	positive	Le gouvernem ent fédéra...	-0.11	0.00	0.00	0.00
		Ottawa to extend temporary...	https://montreal.citynews.ca/2026/03/13...	Mar 13, 2026, 8:48:00 AM	oui	positive	Ottawa extends temporary...	-0.41	0.00	0.00	0.00
		Ottawa to extend temporary...	https://www.thesstar.com/news/canada/...	Mar 13, 2026, 7:56:00 AM	oui	positive	Le gouvernem ent fédéra...	-0.56	0.00	0.15	0.01
	Politique	Ottawa grants 12-month...	https://www.bcc.ca/news/canada/montreal...	Mar 13, 2026, 8:23:00 AM	oui	positive	The federal government has...	-0.33	0.00	0.00	0.00
4	Economie et finances	Le Québec perd 57 000 emplois e...	https://www.journaldequebec.com/20...	Mar 13, 2026, 1:28:25 PM	oui	negative	Le Québec a enregistré...	-0.00	0.00	0.00	0.00
	Travail	Bond surprise du taux de...	https://www.ledevoir.com/economie/96...	Mar 13, 2026, 11:40:00 AM	non	negative	Le Québec a enregistré...	-0.27	0.00	0.00	0.00
		Québec Le taux de chômage...	https://www.lapresse.ca/affaires/econo...	Mar 13, 2026, 12:40:00 PM	non	negative	Le Québec a enregistré...	-0.33	0.00	0.00	0.00
		Hausse du taux de chômage...	https://www.lapresse.ca/affaires/econo...	Mar 13, 2026, 12:40:07 PM	non	negative	Le Québec a enregistré...	-0.33	0.00	0.00	0.00
5	Economie et finances	Coup d'envoi d'un proje...	https://www.journaldequebec.com/20...	Mar 13, 2026, 7:25:26 PM	oui	positive	Lancement d'un projet immobili...	0.86	0.00	0.00	1.00

INTELLIGENCE

Artificial Analysis Intelligence Index; Higher is better



```
=====
Analyzing Quebec City Relevance with Gemini
Config: 5 articles/call, 5 parallel batches
=====
```

Total: 1210 articles → 242 batches

✳ Processing 5 batches in parallel...

- ✓ Batch 2/242 complete (5 articles)
- ✓ Batch 3/242 complete (5 articles)
- ✓ Batch 1/242 complete (5 articles)
- ✓ Batch 5/242 complete (5 articles)
- ✓ Batch 4/242 complete (5 articles)

✳ Processing 5 batches in parallel...

- ✓ Batch 8/242 complete (5 articles)
- ✓ Batch 6/242 complete (5 articles)
- ✓ Batch 7/242 complete (5 articles)
- ✓ Batch 10/242 complete (5 articles)
- ✓ Batch 9/242 complete (5 articles)

✳ Processing 5 batches in parallel...

- ✓ Batch 14/242 complete (5 articles)
- ✓ Batch 15/242 complete (5 articles)
- ✓ Batch 12/242 complete (5 articles)
- ✓ Batch 13/242 complete (5 articles)
- ✓ Batch 11/242 complete (5 articles)

✳ Processing 5 batches in parallel...

- ✓ Batch 16/242 complete (5 articles)
- ✓ Batch 19/242 complete (5 articles)
- ✓ Batch 17/242 complete (5 articles)
- ✓ Batch 18/242 complete (5 articles)
- ✓ Batch 20/242 complete (5 articles)

The screenshot shows the Google Cloud Run console for a job named 'quebec-news-pipeline'. The job is in the 'Failed' state. The console displays a list of execution attempts, with the most recent one failing due to a '429 Client Error: Too Many Requests for url: https://huggingface.co/api/models/ca...'. The console also shows the job's configuration, including the region 'us-central1' and the last updated time 'Feb 8, 2026, 10:25'.

```
> 2026-02-10 12:31:50.544 EST 2026-02-10 17:31:50,542 [INFO] =====
> 2026-02-10 12:31:50.544 EST 2026-02-10 17:31:50,542 [INFO] STAGE 2: GEMINI RELEVANCE ANALYSIS
> 2026-02-10 12:31:50.544 EST 2026-02-10 17:31:50,542 [INFO] =====
> 2026-02-10 12:31:51.933 EST 2026-02-10 17:31:51,933 [INFO] HTTP Request: POST https://aiplatform.googleapis.com/v1beta1/projects/clean-node-480723-k8/locations/global/cachedCont...
> 2026-02-10 12:31:51.934 EST 2026-02-10 17:31:51,934 [INFO] Cache created: projects/265921337680/locations/global/cachedContents/2981659892810514432
> 2026-02-10 12:31:51.935 EST 2026-02-10 17:31:51,935 [INFO] AFC is enabled with max remote calls: 10.
> 2026-02-10 12:31:51.937 EST 2026-02-10 17:31:51,937 [INFO] AFC is enabled with max remote calls: 10.
> 2026-02-10 12:31:51.939 EST 2026-02-10 17:31:51,939 [INFO] AFC is enabled with max remote calls: 10.
> 2026-02-10 12:31:54.331 EST 2026-02-10 17:31:54,330 [INFO] HTTP Request: POST https://aiplatform.googleapis.com/v1beta1/projects/clean-node-480723-k8/locations/global/publishers...
> 2026-02-10 12:32:01.951 EST 2026-02-10 17:32:01,951 [INFO] HTTP Request: POST https://aiplatform.googleapis.com/v1beta1/projects/clean-node-480723-k8/locations/global/publishers...
> 2026-02-10 12:32:01.952 EST 2026-02-10 17:32:01,952 [WARNING] Batch 2 rate limited, waiting 30s (attempt 1)
> 2026-02-10 12:32:31.953 EST 2026-02-10 17:32:31,953 [INFO] AFC is enabled with max remote calls: 10.
> 2026-02-10 12:32:34.859 EST 2026-02-10 17:32:34,859 [INFO] HTTP Request: POST https://aiplatform.googleapis.com/v1beta1/projects/clean-node-480723-k8/locations/global/publishers...
> 2026-02-10 12:34:32.282 EST 2026-02-10 17:34:32,282 [INFO] HTTP Request: POST https://aiplatform.googleapis.com/v1beta1/projects/clean-node-480723-k8/locations/global/publishers...
> 2026-02-10 12:34:32.289 EST 2026-02-10 17:34:32,289 [INFO] AFC is enabled with max remote calls: 10.
> 2026-02-10 12:34:32.291 EST 2026-02-10 17:34:32,291 [INFO] AFC is enabled with max remote calls: 10.
> 2026-02-10 12:34:32.293 EST 2026-02-10 17:34:32,293 [INFO] AFC is enabled with max remote calls: 10.
> 2026-02-10 12:34:34.362 EST 2026-02-10 17:34:34,362 [INFO] HTTP Request: POST https://aiplatform.googleapis.com/v1beta1/projects/clean-node-480723-k8/locations/global/publishers...
```

```
[18]
✓ Os
from gliclass import GLiClassModel
model = GLiClassModel.from_pretrained("knowledgator/gliclass-x-base")
print(model)

... GLiClassModel(
  (model): GLiClassUniEncoder(
    (text_projector): FeaturesProjector(
      (linear_1): Linear(in_features=768, out_features=768, bias=True)
      (act): GELUActivation()
      (dropout): Dropout(p=0.1, inplace=False)
      (linear_2): Linear(in_features=768, out_features=768, bias=True)
    )
    (classes_projector): FeaturesProjector(
      (linear_1): Linear(in_features=768, out_features=768, bias=True)
      (act): GELUActivation()
      (dropout): Dropout(p=0.1, inplace=False)
      (linear_2): Linear(in_features=768, out_features=768, bias=True)
    )
    (pooler): FirstTokenPooling1D()
    (scorer): ScorerDot()
    (dropout): Dropout(p=0.1, inplace=False)
    (encoder_model): DebertaV2Model(
      (embeddings): DebertaV2Embeddings(
        (word_embeddings): Embedding(250104, 768, padding_idx=0)
        (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (encoder): DebertaV2Encoder(
        (layer): ModuleList(
          (0-11): 12 x DebertaV2Layer(
            (attention): DebertaV2Attention(
              (self): DisentangledSelfAttention(
                (query_proj): Linear(in_features=768, out_features=768, bias=True)
                (key_proj): Linear(in_features=768, out_features=768, bias=True)
                (value_proj): Linear(in_features=768, out_features=768, bias=True)
                (pos_dropout): Dropout(p=0.1, inplace=False)
                (dropout): Dropout(p=0.1, inplace=False)
              )
            (output): DebertaV2SelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): DebertaV2Intermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): DebertaV2Output(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
      (rel_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-07, elementwise_affine=True)
    )
  )
)
```

```

***
-----
HTTPError                                Traceback (most recent call last)
/tmp/ipython-input-543656551.py in <cell line: 0>()
      4
      5 # Get sitemap (handles malformed XML)
----> 6 df = adv.sitemap_to_df("https://www.tvanouvelles.ca/news.xml")
      7
      8 # Filter past 24 hours

-----
6 frames
/usr/lib/python3.12/urllib/request.py in http_error_default(self, req, fp, code, msg, hdrs)
    637 class HTTPDefaultErrorHandler(BaseHandler):
    638     def http_error_default(self, req, fp, code, msg, hdrs):
--> 639         raise HTTPError(req.full_url, code, msg, hdrs, fp)
    640
    641 class HTTPRedirectHandler(BaseHandler):

HTTPError: HTTP Error 403: Forbidden

```

Figure 5: HTTP 403 Forbidden error from news sites

```

▼<item>
  <title>Samantha Brown Calls This Winter Wonderland North American City 'A Snow Globe Come
  <link>http://www.bing.com/news/apiclick.aspx?ref=FexRss&aid=&tid=692f41434eb440dca9153098e
  life%2far-AA1RpdS0&c=18365263743787097350&mkt=pt-br</link>
  <description>Few things are prettier than snow-capped building and trees, and a trip into
  <pubDate>Sat, 29 Nov 2025 14:28:00 GMT</pubDate>
  <News:Source>Islands.com on MSN</News:Source>
  <News:Image>http://www.bing.com/th?id=OVFT.NSR26SShb1RR9ikczcDkxC&pid=News</News:Image>
  <News:ImageSize>w={0}&h={1}&c=14</News:ImageSize>
  <News:ImageMaxWidth>700</News:ImageMaxWidth>
  <News:ImageMaxHeight>393</News:ImageMaxHeight>
</item>

```